

C++23 失败语义工程手册

Modern C++ Failure Semantics Handbook

CPP-FAILURE-MODEL

v1.0.0

PUBLICATION EDITION

权威内容来源为 Markdown。本件为 v1.0.0 正式阅读版。发布身份与校验摘要见随附发布记录。

阅读与版本说明

PUBLICATION EDITION · 正式阅读版 · v1.0.0

本件从已接受的 Markdown 冻结提交取源。正文中的“待审核”“PDF NOT BUILT”等保留提交时的历史文字；内容接受不等于 PDF 或正式发布批准。标题、章节编号、代码与审查模板 不因出版改写。封面、目录和代码续页标签是生成的阅读辅助。

制品标识 CPP-FAILURE-MODEL

版本 1.0.0

状态 PUBLICATION EDITION

阅读密度 balanced · Visual Profile v1.0, 已冻结；出版版本 v1.0.0

基准 commit 45b305eace0f057420587d685cb3f962f3f0552c

准备标识 c79dfba6f1c447c96adac7d4

使用方式 目录与书签可跳转；合订版中的核心文档链接跳转到本件对应篇章。独立版在源字节匹配时提供固定 commit 链接，否则指向随附的 source 快照。11 份正文逐字节绑定 45b305e；review 目录不进入正文。出版配置的实际字节以准备记录为准。正文原有编号不重新连续编号。性能、并发动态检测、完整协程、ABI 和分布式实验不因出版升级为已验证。

FM-GUIDE source@45b305e ACCEPTED CONTENT

FM-0 source@45b305e ACCEPTED CONTENT

FM-1 source@45b305e ACCEPTED CONTENT

FM-2 source@45b305e ACCEPTED CONTENT

FM-3 source@45b305e ACCEPTED CONTENT

FM-4 source@45b305e ACCEPTED CONTENT

FM-5 source@45b305e ACCEPTED CONTENT

FM-6 source@45b305e ACCEPTED CONTENT

FM-7 source@45b305e ACCEPTED CONTENT

FM-8 source@45b305e ACCEPTED CONTENT

FM-9 source@45b305e ACCEPTED CONTENT

目录

阅读与版本说明	2
FM 系列阅读约定：术语、状态与证据	6
1. 一次操作，而不是一个语言特性	6
2. 统一词义与主讲位置	7
3. 沿状态转换审查，而不是只标注“可能 throw”	9
4. 公共审查合同与章节增量	9
5. 技术论述与证据的阅读规则	10
FM-0 · 统一失败模型	11
阅读入口	11
本章目录	11
一、操作与基本词义	12
二、责任分类	15
三、检测、传播与恢复	19
四、边界与影响范围	25
五、相互独立的分析维度	28
六、把模型用于实际场景	31
七、接口合同与反例	35
八、系统状态与副作用	38
九、项目交接与回查	40
FM-1 · 契约与可信边界	46
阅读入口	46
本章目录	46
一、合同与责任	46
二、验证边界与断言	51
三、检查访问与未定义行为	56
四、终止与不可达承诺	60
五、可信表示与类型设计	64
六、反例与回查	67

FM-2 · 值通道与错误类型	71
阅读入口	71
本章目录	71
一、结果空间与表示	71
二、错误类型与传播	74
三、组合条件与状态责任	77
四、成本、反例与回查	81
FM-3 · 异常传播语义	84
阅读入口	84
本章目录	84
一、抛出对象与匹配	84
二、展开、重抛与中间层	86
三、捕获与边界	89
四、选型、成本与回查	92
FM-4 · 资源清理与状态保证	96
阅读入口	96
本章目录	96
一、所有权与保证语言	96
二、事务式状态转换	98
三、清理与外部副作用	102
四、审查与回查	104
FM-5 · 异常边界与泛型保证	105
阅读入口	105
本章目录	105
一、传播边界与表达式	105
二、移动与值复制	107
三、容器策略与类型条件	109
四、提交与组合保证	111
五、审查与回查	113
FM-6 · 生命周期与资源失败	115
阅读入口	115

本章目录	115
一、构造与对象有效性	115
二、分配与资源压力	118
三、显式完成和清理	120
四、审查与回查	122
FM-7 · 系统错误与领域转换	123
阅读入口	123
本章目录	123
一、原生错误身份	123
二、通道、领域与上下文	126
三、审查与回查	128
FM-8 · 执行边界与恢复协议	130
阅读入口	130
本章目录	130
一、线程与异步观察	130
二、取消、超时与关闭	134
三、集成和远端边界	136
四、恢复协议与影响范围	137
五、审查与回查	140
FM-9 · 项目失败契约与验证	141
阅读入口	141
本章目录	141
一、项目选择与责任	142
二、通道与错误域政策	144
三、状态、所有权与边界政策	147
四、恢复与可观测性政策	149
五、验证工具与证据	152
六、可裁剪模板与回查	153
Modern C++ Failure Model — 最终总图	157

FM 系列阅读约定：术语、状态与证据

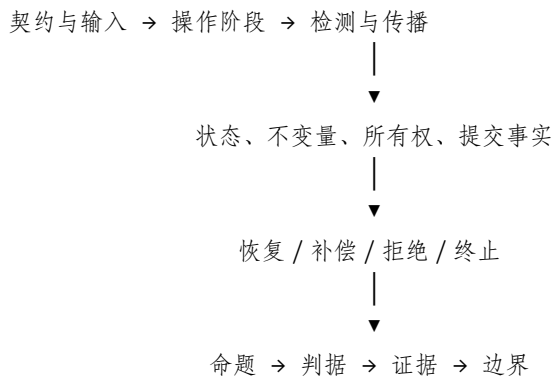
SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

本文件与 **FM** 总导航共同服务 C++23 失败语义工程手册。它确定系列内用语和主讲位置，不取代标准、项目合同或正文中的适用条件。十篇文件名、旧 § 编号与旧锚点保持；新增主题分组用于阅读，不改变历史审校对象身份。

1. 一次操作，而不是一个语言特性

分析对象是“在满足前置条件的输入上执行一次操作”。先明确成功后置条件（**success postcondition**），再问可能在哪里失去成功、如何报告、哪些状态已变化，以及谁有资格恢复。故障注入可以检查某些路径，但“如何证明”不能一律理解为跑一次样例。



正常返回不等于业务成功：返回 **unexpected**、**false** 或合法的 **absence** 都可能是接口规定的正常返回分支。成功与各失败分支须分别写后置条件；违反前置条件也不能当成被调函数承诺处理的普通失败输入。

2. 统一词义与主讲位置

2.1 原因、状态、表示与传播

术语	本系列含义	主讲位置
失败 (failure)	操作未达到其正常成功后置条件；可能仍正确履行“报告失败”的接口合同	FM-0 §2
领域结果 (domain outcome)	合同允许的普通分支；例如 lookup miss，不必归为组件失败	FM-0 §5
故障原因 (fault)	导致偏差的原因；不自动等于已观察到的对外失败	FM-0 §3
错误状态 (error state)	系统状态偏离所要求的正确状态；可能被纠正而未向外失效	同上
错误表示 (error representation / error object)	错误码、错误对象等机器可处理信息；对象本身可以完全有效	FM-2 §6、FM-7 §3
异常 (exception)	C++ 异常对象及其非局部传播机制，不是失败原因分类	FM-3 §1-2
契约违反 (contract violation)	调用方或实现破坏约定；具体后果须看接口与语言规则，不能一概等同 UB	FM-1 §6
终止 (termination)	不再沿普通调用返回继续执行的处置；不是携带给调用者的错误值	FM-1 §33

`error` 在可靠性讨论中可能指 `error state`，在 API 中又常指 `error object`。本系列保留这两种语境并写出限定词，不把原有 `fault` → `error state` → `failure` 模型强行改成“`error` 只能是一个返回值”。`panic` 不是 C++23 的统一机制；提及时须注明语言 / 运行时和具体行为，不将其默认等同于 `throw` 或 `terminate`。

2.2 清理、回滚与恢复

术语	要回答的不同问题	主讲位置
清理 (cleanup)	谁释放 / 结束已取得资源的责任? 清理是否会失败?	FM-4 §1-3
回滚 (rollback)	能否恢复到合同指定的先前状态? 逆操作自己失败怎么办?	FM-4 §10
恢复 (recovery)	是否重新进入能继续承担职责的定义良好状态?	FM-0 §16
补偿 (compensation)	已发生的不可撤销效果如何用后续动作补救? 不等于原操作从未发生	FM-4 §15
重试 (retry)	再执行是否安全、仍有预算且被策略允许?	FM-8 §18-19
恢复边界 (recovery boundary)	哪一层有足够状态、上下文和权限决定下一步?	FM-0 §17
传播边界 (failure boundary)	当前错误形式不能或不应越过哪里?	FM-8 §1
失败域 (failure domain)	哪些任务、资源或组件会受到影响?	FM-0 §20

本系列中文“传播边界”对应正文的 **failure boundary**；“错误域”对应 **error category/domain**，与失败影响范围不是同一概念。**catch** 是处理动作，既不证明已经恢复，也不保证报告动作成功。

2.3 保证不是一条从失败到终止的等级线

状态保证统一回查 [FM-4 §4](#)。本系列分别描述三条轴：

分析轴	可用表述	不能推出什么
失败后状态	strong : 约定状态不变； basic : 不变量 / 资源管理成立、值可变；明确部分进度；缺少可依赖保证	basic 不意味着下一次操作一定成功；部分进度可以是有效状态
异常传播	no-throw ； noexcept 规格；某个完整表达式的查询结果	不保证 no-fail 、无错误返回、正常完成或完整清理
处置与可恢复性	拒绝、恢复、补偿、升级、重启、终止	“不可恢复”不是 strong/basic 旁边的状态保证等级

这里的 **no-fail** 指合法前提和约定模型内确实完成操作；终止不算完成。**Strong guarantee** 必须给出受保护状态及失败通道，不能默认覆盖日志、文件、网络和远端世界。**effects unspecified** 不是自动降级为 **basic**，也不自动等同 **UB**。

RAII 将资源清理责任与对象生命周期绑定；适用的是正常退出和确实发生的展开，不承诺所有终止都执行析构。**noexcept** 内部可抛出并捕获；搜索 **handler** 触及非抛出边界才触发相应终止规则。参见 [N4950](#) 异常规格与终止及展开限制。

3. 沿状态转换审查，而不是只标注“可能 throw”

提交点（commit point）指合同把准备状态接受为新状态的边界；它需要实现依据，不是给某行代码加上名字就存在。主讲及状态表见 FM-4 §7。

每次 mutation 依次记录：

1. **S0：进入前。**哪些前提、不变量、值和所有权成立？观察范围是什么？
2. **S1：准备中。**分配、校验、构造、回调、日志可能失败；是否已经碰过受保护状态？
3. **S2：提交后。**哪些新状态已生效，旧状态由谁清理？返回或通知还能否失败？
4. **调用方观察。**成功、明确未提交、明确部分提交、已提交但报告失败、完成状态未知分别如何表示？

若准备阶段修改了外部状态，就不能仅凭“临时对象会析构”声称失败前原状不变。若提交后报告失败，也不能让调用者按“失败一定未提交”盲目重试。内存中的 swap、文件操作和数据库提交具有不同观察 / 持久化边界，不因都称 commit 就共享保证。

数组填充问题的分析方式。对已有的 fill(buffer) 类接口，先约定是在给存活元素赋值，还是在原始存储中构造。再说明成功计数、失败后的已提交前缀、当前元素、剩余元素和资源责任；分配、T 的构造 / 赋值、回调与错误对象构造都要纳入失败集。没有实现与这些前提，仅凭“参数是数组”或“返回错误值”不能断言无异常。此处是合同分析，不新增一个未经验证的填充实现。

4. 公共审查合同与章节增量

全系列共用以下八项问题。各章末尾的 checklist 只承担主题增量；不必让十篇分别复制完整表。

项	必须写清的内容	证据应对准的命题
C1 输入与成功	合法前提、正常领域结果、成功后置条件	输入与结果判断不是同一个模糊 bool
C2 失败集与通道	来源、检测点、值 / 异常 / 终止、错误转换	所有声明通道及转换失败都有归属
C3 状态与提交	观察范围、每个阶段、不变量、失败后置条件、提交认知	失败发生后还能依赖什么，而不只是 catch 到了什么
C4 所有权与清理	取得、转移、失败返还 / 销毁、析构及显式完成	无泄漏不等于回滚，cleanup 不等于业务成功
C5 组合前提	参数、临时对象、T/E、allocator、回调、返回及报告	no-throw 与 no-fail 分别成立于哪些条件
C6 恢复与影响范围	谁决定恢复 / 补偿 / 终止，失败域与并发观察点	处理失败的代码自身仍满足合同
C7 重试与控制状态	幂等 / 去重、截止期、预算、取消、完成不确定性	请求停止不等于已经停止，超时不证明未提交
C8 验证与限制	标准 / 工程建议 / 观测、oracle、反例、环境、SKIP、未测项	测试能区分承诺与错误实现，结果不被扩大

无关项可以说明不适用。局部纯函数不必虚构远端重试政策；项目模板也不强迫所有 API 采用同一错误类型。可填写模板统一放在 FM-9 §26。

5. 技术论述与证据的阅读规则

先看章节用途、主阅读线和失败契约，再读机制 / 例子、状态影响、边界条件与章末回查。不机械要求每篇都有同样十五个标题，也不增加没有证据的 Rust/Zig 对照。

标准规则继续引用 N4950 固定条款；“建议”“默认”表示工程选择；运行结果只绑定实际工具链和程序；状态表中的预期是推理目标，不伪装成执行输出。无 `fm-test` 标记的 C++ 块不因排版更专业而成为完整实验，危险上下文和省略条件继续有效。

验证分开报告：文档结构 / 链接、C++ 编译与负例诊断、性能观察、并发动态检测。旧 JSON 不更新为新正文摘要；本轮另比较实际提取源码和判据，新增执行结果另存。20 个样例覆盖若干语言与库边界，不是全部片段、异常注入点或恢复协议的证明。

本轮仅整理 Markdown 与相应审计材料。长代码、表格和流程的分页留给未来出版层，不插入私有 `renderer` 指令；重要内容不藏进 `<details>`。PDF **NOT BUILT / NOT VALIDATED**，G 系列与已发布制品不受本次 FM 状态改变影响。

FM-0 · 统一失败模型

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [下一章：FM-1 · 术语与审查约定](#)

阅读入口

建立操作、结果、状态、恢复权限与失败域的共同模型。值通道、异常规格及生命周期规则交给各自主讲章，不在总论重复定义。

主阅读线。§0-8 → §14-20 → §28-33 → §40-47；其余为系统场景回查。

失败契约。先给成功条件，再分别登记故障来源、错误表示、状态后置条件与处置。一个 `ParseError` 可以完全有效；一个 `catch` 也可以没有完成恢复。

证据边界。本章采用合同推理；POSIX `open` 的定向 T20 在样例附录，不是配置加载事务的完整测试。无 `fm-test` 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、操作与基本词义
- 二、责任分类
- 三、检测、传播与恢复
- 四、边界与影响范围
- 五、相互独立的分析维度
- 六、把模型用于实际场景
- 七、接口合同与反例
- 八、系统状态与副作用
- 九、项目交接与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、操作与基本词义

0. 文档定位

本文建立一套适用于现代 C++ 系统工程的统一失败模型，用于回答：

- 什么是失败？
- 哪些情况属于正常结果，哪些是真正错误？
- 失败由谁检测？
- 应该如何表示和传播？
- 哪一层有资格决定如何恢复？
- 失败以后对象和系统处于什么状态？
- 一个失败应该影响多大范围？
- 什么情况下应该继续运行，什么情况下应该终止？

本文不试图直接回答：

应该使用 `exception` 还是 `std::expected`？

因为这不是失败设计的第一问题。

正确顺序是：

```
理解 failure
  ↓
明确责任
  ↓
明确状态保证
  ↓
明确恢复边界
  ↓
明确 failure domain
  ↓
最后选择 transport mechanism
```

因此：

```
exception
std::expected
std::optional
std::error_code
assert
std::terminate
```

都只是更大 Failure Model 中的不同工具。

1. 核心原则

现代 C++ 不存在单一错误处理机制。

一个完整 Failure Model 至少需要描述：

```
Preconditions
Success semantics
Failure taxonomy
Detection
Representation
Translation
Propagation
Handling
Recovery
State guarantee
Resource guarantee
Recovery boundary
Failure domain
Terminal behavior
```

因此一个函数的真实接口绝不只是：

```
Result operation(Input input);
```

而是：

```
operation contract
=
input contract
+ success contract
+ failure contract
+ state contract
+ resource contract
```

2. Failure 不等于 Exception

这是整个模型最重要的概念分离之一。

2.1 Failure

failure 描述：

操作未达到当前抽象层约定的正常成功后置条件。

接口若明确允许报告失败，返回错误值仍可正确履行接口合同，不代表被调实现违约。正常领域结果是否算失败也由合同决定，不能仅由“没有值”判断。

例如：

```
parse_frame(bytes)
```

如果输入格式非法而无法产生 Frame: parse failure

这是 failure。

2.2 Exception

exception 是 C++ 的一种: control-flow + failure propagation mechanism

例如:

```
throw ParseError{};
```

其中: ParseError

描述失败,

而:

```
throw
```

描述失败如何传播。

因此: failure $\neq$ exception

同一个 failure 可以表示成:

```
bool
enum
std::optional<T>
std::expected<T, E>
std::error_code
exception
```

若策略不能继续恢复, 还可能进入终局处置 (不是另一种返回值表示):

```
std::terminate()
```

3. Fault / Error / Failure

在工程讨论中建议区分三个层次。

3.1 Fault

导致错误的原因。

例如:

```
错误的长度计算
错误的状态转换
磁盘损坏
网络链路中断
```

3.2 Error

这里的 error 指**错误状态 (error state)**; API 中的错误码 / 错误对象则称**错误表示 (error representation)**。一个完全有效的 ParseError 对象可以报告外部输入拒绝, 并不说明解析器自身处于损坏状态。两种语境统一见[系列术语](#)。

系统内部状态已经偏离正确状态。

例如设计要求: $0 \leq \text{index} < 16$

但内部产生: $\text{index} = 27$

这是错误状态。

3.3 Failure

错误最终影响到组件对外承诺。

例如：

```
decode_message()
```

无法返回合法结果。

可以粗略理解：

```
fault
  ↓
error state
  ↓
failure
```

但不要假设三者必然一一对应。

错误可能被内部纠正而永远不会发展为 **failure**。

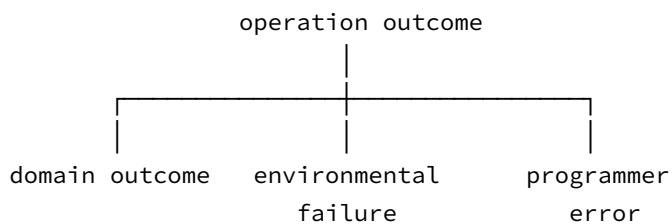
二、责任分类

4. 第一维：失败责任分类

在选择错误机制之前，首先判断：

这个结果为什么会发生？

推荐将运行时情况首先分为三类：



5. Domain Outcome

有些“没有得到值”的情况并不是错误。

例如：

```
std::optional<User> find_user(UserId id);
```

可能存在：

```
user exists
  → User

user does not exist
  → no value
```

用户不存在可能完全属于领域模型正常状态。

其他典型例子：

```
cache miss
map lookup miss
queue empty
EOF
pattern not found
try_lock failed
```

这些情况通常：

- 是预期的；
- 可能频繁发生；
- 调用者自然需要分支处理。

因此：

不要把正常领域状态强行建模成异常。

5.1 optional 的语义

一个良好的第一近似是：

```
std::optional<T>
≈
T or absence
```

例如：

```
std::optional<User> find_user(UserId id);
```

表达：

```
存在 User
或者
不存在 User
```

它通常不应该同时承担：

```
数据库断开
权限失败
数据损坏
```

否则：absence

和：failure

会被压缩为同一个状态。

6. Environmental Failure

环境失败是程序本身逻辑正确，但外部世界无法满足操作要求。

例如：

```
file not found
disk full
permission denied
connection reset
timeout
invalid packet
database unavailable
broker unavailable
remote service rejected request
```

这些属于：runtime operational failure

而不是：program bug

例如：

```
std::expected<File, OpenError>
open_file(std::string_view path);
```

可能产生：

```
enum class OpenError {
    not_found,
    permission_denied,
    too_many_open_files,
    io_error,
};
```

调用者可以根据错误采取：

```
retry
fallback
skip
create
report
shutdown
```

7. Programmer Error

第三类完全不同。

例如：

```
Message& message_at(std::size_t index) {
    assert(index < messages_.size());
    return messages_[index];
}
```

如果系统设计已经保证：index 必须来自合法 message table
那么越界意味着：internal invariant violation
而不是普通运行时失败。

典型 programmer error 包括：

```
违反 API 前置条件
内部索引越界
非法状态机转换
double ownership
double free
使用失效引用
破坏类不变量
到达逻辑上不可能的状态
```

这类问题一般不应该被包装成普通业务错误然后继续运行。

8. 表面相同的错误可能属于不同类别

例如：index out of range

本身无法决定应该如何处理。

外部输入

```
parse_packet(untrusted_bytes);
```

packet 内声明：index = 1000

但 packet 实际只有 10 个元素。

这是：invalid external input

应该：

```
validate
→ report failure
```

内部索引

```
auto index = generated_table[id];

assert(index < messages.size());
```

如果生成表保证 index 合法，那么失败说明：program invariant broken

因此可能：assert / fail-fast

原则

不要问：

这个错误看起来严重吗？

而应该问：

调用者是否有责任保证这种情况永远不会出现？

这是判断：

```
recoverable error  
vs  
contract violation
```

的重要标准。

三、检测、传播与恢复

9. Detection / Representation / Translation / Propagation

失败处理不是一个动作，而是一条生命周期。

推荐显式拆分：

```
detect  
↓  
represent  
↓  
translate  
↓  
propagate  
↓  
handle  
↓  
recover
```

10. Detection

Detection 回答：

谁最先知道操作无法继续？

可能是：

```
CPU
kernel
system call
allocator
runtime
third-party library
parser
domain logic
invariant check
```

例如：

```
::open(...)
```

打开不存在的文件。

最早发现问题的是：OS / system-call layer

11. Representation

失败被某种值或机制表示。

例如 POSIX：ENOENT

进入 C++ 层以后可能成为：

```
std::error_code
```

再进入配置领域：

```
enum class ConfigError {
    missing,
    permission_denied,
    malformed,
};
```

所以同一失败可能经历：

```
ENOENT
↓
std::errc::no_such_file_or_directory
↓
ConfigError::missing
```

12. Error Translation

translation 是：

从一个抽象层的错误语言，转换成另一个抽象层真正理解的错误语言。

例如：

```

TCP ECONNRESET
  ↓
DatabaseError::connection_lost
  ↓
RepositoryError::unavailable

```

业务逻辑通常不应该依赖：ECONNRESET

因为：TCP

可能只是 repository 当前实现细节。

12.1 Translation 原则

只有当：

抽象层发生变化，并且错误语义也随之变化

才值得 translation。

不要机械制造：

```

ErrorA
  ↓
ErrorB
  ↓
ErrorC
  ↓
ErrorD
  ↓
ErrorE

```

否则错误模型本身会成为新的复杂系统。

13. Propagation

发现失败的层不一定知道如何处理。

例如：

```

std::expected<std::string, FileError>
read_file(std::string_view path);

```

底层知道：file missing

但不知道：

```

应该创建文件吗？
应该使用默认值吗？
应该退出程序吗？
这是 production 还是 development？

```

因此它的职责可能只是：propagate failure

即：

我告诉上一层发生了什么，
但不替上一层决定策略。

14. Handling 不等于 Recovery

这是大型系统中非常重要的区别。

考虑：

```
try {  
    update_state();  
} catch (const std::exception& e) {  
    log(e.what());  
}  
  
continue_processing();
```

这里只能确定：exception was observed

不能确定：system recovered

因为 update_state() 可能已经：

```
update A  
update B  
fail while updating C
```

形成：partial state

15. Handling

Handling 表示：

当前层接受了这个 failure，不再继续按原样向上传播。

例如：

```
auto result = load_config();  
  
if (!result) {  
    log(result.error());  
    return EXIT_FAILURE;  
}
```

这里 main 处理了这个错误。

但它没有恢复。

16. Recovery

Recovery 的标准更高：

系统重新进入一个定义良好、可以继续履行职责的状态。

例如：

```
auto config = load_config();

if (!config && config.error() == ConfigError::missing) {
    config = make_default_config();
}
```

如果成功：

```
missing config
  ↓
fallback
  ↓
valid configuration
  ↓
normal execution
```

这才是真正恢复。

16.1 Recovery 必须知道状态

一个层如果不知道：

```
哪些修改已经发生
哪些资源已经获得
哪些对象仍然有效
哪些操作可以 rollback
当前 invariant 是否仍成立
```

它通常没有资格声称自己已经恢复。

17. Recovery Boundary

这是 Failure Model 最重要的工程概念之一。

假设：

```
main
├─ Application
│   └─ ConfigService
│       └─ ConfigRepository
│           └─ FileReader
│               └─ OS
```

最底层得到：ENOENT

FileReader

知道：file does not exist

不知道：这个文件是否必须存在

ConfigRepository

知道：这是 configuration

但可能仍不知道：production 和 development 是否策略不同

Application

可能知道：

```
production:
    missing config → startup failure

development:
    missing config → generate default
```

因此：Application

才是真正的：recovery boundary

18. Recovery Boundary 原则

一个极其重要的工程原则：

Detect failure as low as necessary; recover as high as necessary.

中文可以理解为：

失败应该在最接近事实的位置检测，在最接近策略的位置恢复。

因此通常：

```
who detects
≠
who decides
```

19. 不要让底层擅自决定 Retry

例如：

```
fetch_from_server();
```

底层发生：timeout

网络层可能不知道：

```
请求是否幂等
整体 deadline 是多少
是否已经在 retry
是否正在 shutdown
重复发送是否危险
业务是否允许 fallback
```

因此：retry

不是单纯错误处理。

它是：recovery policy

应该由拥有足够上下文的一层决定。

四、边界与影响范围

20. Failure Domain

Recovery Boundary 回答：

谁负责决定怎么办？

Failure Domain 回答另一个问题：

这次失败最多应该影响多大的范围？

例如：

```
process
├─ request A
├─ request B
├─ request C
└─ background worker
```

发生一个错误，不一定意味着整个进程都应该失败。

21. Request-Level Failure Domain

例如：malformed HTTP request

合理影响范围可能只是：one request

因此：

```
request fails
service continues
```

22. Task-Level Failure Domain

线程池：

```
worker
├─ task A
├─ task B
└─ task C
```

task B 失败：task B failed

如果任务彼此隔离，则：worker continues

23. Subsystem-Level Failure Domain

例如: `telemetry subsystem`

可能允许: `subsystem restart`

而核心服务继续运行。

24. Process-Level Failure Domain

如果发生:

```
heap corruption
global invariant violation
ownership corruption
impossible global state
```

继续执行可能已经没有可靠语义。

此时: `process termination`

可能比:

```
catch(...)
continue
```

更正确。

25. Recovery Boundary 与 Failure Boundary

这两个概念不要混淆。

Recovery Boundary

回答: 谁有足够上下文选择恢复策略?

Failure Boundary

回答: 失败传播不能以当前形式越过哪里?

典型 failure boundary:

```
thread entry
C ABI
plugin ABI
RPC
process
service
IPC
```

26. C ABI Boundary

例如：

```
extern "C" int plugin_entry() {
    try {
        run_plugin();
        return 0;
    } catch (...) {
        return -1;
    }
}
```

这里 `exception` 必须被转换：

```
C++ exception
    ↓
boundary
    ↓
C-compatible error representation
```

因为另一侧没有 C++ `exception model`。

27. Thread Boundary

异常逃出线程初始函数会调用 `std::terminate()`。若工程目标是把 `worker` 失败限制在线程任务内，边界必须覆盖工作路径和错误报告路径；仅有 `catch (...)` 还不够。[N4950: except.terminate](#)

以下是契约片段：只在线程内保存异常，正常 `join()` 返回后由协调者观察。这里省略了 `run_worker` 的实现以及线程创建、`join` 和协调者报告失败的上层策略。

```
std::exception_ptr failure;
std::thread worker([&failure] {
    try {
        run_worker();
    } catch (...) {
        failure = std::current_exception();
    }
});
worker.join(); // 正常返回后，协调者才能读取 failure。
```

不要在这个 `catch` 内调用契约未知的 `report_failure()`：格式化、日志分配或报告动作若再次抛出，仍可能造成进程终止。只给它加 `noexcept` 也不是恢复方案。

完整样例、异常捕获可能分配的限制和报告失败后备策略，见 [FM-8 §2](#)。

五、相互独立的分析维度

28. 第二维：Failure Representation

错误表示与传播通道要分开看：`bool`、`sentinel`、`enum`、`optional`、`expected` 和 `error_code` 可以携带普通返回分支；`exception` 通过异常控制流传播对象；`termination` 是终局处置，不是返回给调用者的 `transport`。

例如：

```
std::expected<void, Error> update();
```

返回类型不能推出“失败时状态不变”。表示方式主讲见 [FM-2](#)，异常传播见 [FM-3](#)，状态保证统一见 [FM-4 §4](#)。

29. 第三维：State Guarantee

失败后的状态应与异常传播、终止策略分别说明：

分析维度	应回答的问题
失败后状态	strong （可观察状态不变）、 basic （不变量成立但值可变）、明确的部分进度，还是没有可依赖的状态保证？
异常传播	是否允许异常越过 API 边界？ <code>noexcept</code> 只约束这一项。
终局处置	是否拒绝当前操作、升级处理、重启或终止进程？

这些不是一条从弱到强的等级序列。一个返回错误码的 `noexcept` 操作仍可能失败；终止也不等于“成功完成清理”。术语和提交条件统一见 [FM-4 §4 ~ §5](#)。

30. Strong Guarantee

Strong guarantee 针对合同规定的失败通道和可观察状态：失败时该状态与调用前一致。成功后提交新状态；常见实现先准备隔离的新状态，再执行真正可完成的提交，并检查后续返回与清理。

这不是对整个外部世界的自动原子性承诺。完整条件、反例和提交状态表统一见 [FM-4 §5–7](#)，本章不再另设一版定义。

31. Basic Guarantee

Basic guarantee 表示失败后不变量和资源管理仍成立，但值可以改变；哪些后续操作仍合法由具体合同决定。不能从“对象仍能析构”单独推出整个组件仍可正常服务。主说明见 [FM-4 §4](#)。

32. No Useful Guarantee

No useful guarantee 表示关注的失败路径缺少可依赖的状态承诺，不能自行补成 **basic**。它既不是一种错误表示，也不把所有此类路径自动判为 **UB**；具体语言 / 库条款仍须分别检查，见 [FM-4 §4](#) 和 [FM-5 §11](#)。

33. State Guarantee 与 Error Transport 正交

例如：

```
std::expected<void, Error> update();
```

可能提供：strong guarantee

也可能只提供：basic guarantee

甚至可能糟糕到：partial invalid state

同样：

```
void update();
```

虽然通过 exception 失败，也可以提供：strong guarantee

因此不要建立错误对应：

```
expected = safe
exception = unsafe
```

真正应该分析的是：

```
transport mechanism
×
state guarantee
```

34. 第四维：Failure Frequency

常见、少见等发生频率会影响机制成本，但应根据实际工作负载判断，不能由错误名称推断。

频率与严重性独立：高频失败也可能严重，罕见结果也可能只是普通拒绝。§35 ~ §36 讨论频率；§37 的 catastrophic 是严重性与处置问题，不是第三档发生频率。

35. Common Outcome

例如：

```
lookup miss
queue empty
EOF
cache miss
try_lock failure
```

通常应该具有：

```
low control-flow cost
explicit semantics
easy branch handling
```

常见表示：

```
bool
enum
optional
expected
```

36. Rare Failure

例如：

```
configuration unavailable
rare I/O failure
deep initialization failure
```

如果失败确实稀有，并且需要跨多层传播：exception

可能成为合理设计。

但频率只是决策输入之一，不是唯一标准。

37. Catastrophic Failure

这里切换到严重性与终局处置维度，不继续给发生频率分档。

堆损坏、中心不变量失效、重复所有权等问题，可能使进程状态不再可信。工程上通常应考虑 fail-fast，而不是假装通过普通错误值恢复；适用范围仍取决于损坏的失败域和系统隔离能力。

这是工程处置建议，不是说所有严重错误都由同一种语言机制报告，也不是说 terminate 会执行完整资源清理。

38. 第五维：Resource Guarantee

状态正确不只是业务字段正确。

还必须保证：

```
memory
file handles
locks
sockets
transactions
temporary files
GPU resources
OS handles
```

在失败路径中受到正确管理。

这就是 RAII 对 C++ Failure Model 的基础意义。

Failure contract 应明确：

失败以后：
 是否存在资源泄漏？
 锁是否释放？
 所有权是否清晰？
 temporary resource 是否清理？

39. RAII 的 Failure Model 含义

RAII 将资源所有权绑定到对象生命周期，使正常作用域退出与**确实发生的栈展开**共用析构清理。

acquire → owner → work → normal exit / unwinding → destructor

不能把这条路径扩展到所有终止：无匹配 handler 或触及 noexcept 边界时，不能依赖完整栈展开。清理规则与终止边界的主说明见 [FM-3 §5](#)。

工程上还要审查析构的可失败操作；RAII 不自动提供持久化提交、业务回滚或“资源关闭一定成功”的保证。

六、把模型用于实际场景

40. Complete Failure Pipeline

综合起来，一个 failure 可以经历：

```

source / fault
    ↓
detection
    ↓
representation
    ↓
translation
    ↓
propagation
    ↓
recovery boundary
    ↓
recovery policy
    ↓
state transition
    ↓
failure domain containment
  
```

任何复杂系统错误处理设计，都应该能够映射回这条链。

41. 示例：文件读取

契约片段：

```

std::expected<Config, ConfigError>
load_config(std::string_view path);
  
```

以 POSIX `open` 打开已有配置文件为例，返回值与错误指示必须分开：

```
open(path, O_RDONLY) returns -1
↓
capture errno immediately
↓
missing-path error: saved errno == ENOENT
↓
translate to ConfigError::missing
```

不是 `open()` “返回 `ENOENT`”。成功返回文件描述符；失败返回 `-1` 并设置 `errno`。`O_CREAT` 等选项会改变语义，不能把“目标文件不存在”无条件等同于打开失败。**open(2)**：返回值与错误

工程模型仍为：

Detection	OS
Representation	saved errno / error_code
Translation	ConfigError::missing
Propagation	load_config returns unexpected
Recovery boundary	Application startup
Recovery	development → default config production → fail startup
Failure domain	application startup
State guarantee	existing configuration remains unchanged

最后一项依赖先完整读取、解析和验证，再提交新配置，不能仅由返回 `expected` 推出。相关错误转换见 **FM-7 §1**。

42. 示例：Binary Parser

```
std::expected<Frame, ParseError>
parse_frame(std::span<const std::byte> input);
```

可能失败：

```
truncated input
invalid length
checksum mismatch
unknown version
```

模型：

Responsibility	external input
Frequency	depends on data quality; invalid input is expected operationally
Detection	parser

FM-0 · 42. 示例: Binary Parser · continued

```

Representation
    ParseError

Propagation
    expected

Recovery boundary
    message processor

Recovery
    drop / count / quarantine / report

Failure domain
    one frame

State guarantee
    input unchanged
    no externally visible partial Frame

```

这是非常适合 value-based error handling 的类型。

43. 示例: 内部 Decode Table

```

const DecodePlan& plan_for(std::size_t index) {
    assert(index < plans.size());
    return plans[index];
}

```

假设 `index` 来源完全受内部控制。

模型:

```

Responsibility
    programmer

Failure type
    invariant violation

Recovery
    usually none locally

Representation
    assertion / fatal check

```

FM-0 · 43. 示例：内部 Decode Table · continued

Failure domain
process or subsystem

Reason
continuing would hide a programming defect

44. 示例：数据处理 Pipeline

```
Kafka
  ↓
download
  ↓
decompress
  ↓
parse
  ↓
decode
  ↓
produce
```

假设 gzip 数据损坏。

Detection

decompression library

Low-level representation

DecompressError::invalid_stream

Translation

ProcessError::corrupted_capture

Propagation

worker processing result

Recovery boundary

worker / job coordinator 知道：

```
是否 retry
是否 dead-letter
是否标记失败
是否 ACK
```

Failure domain

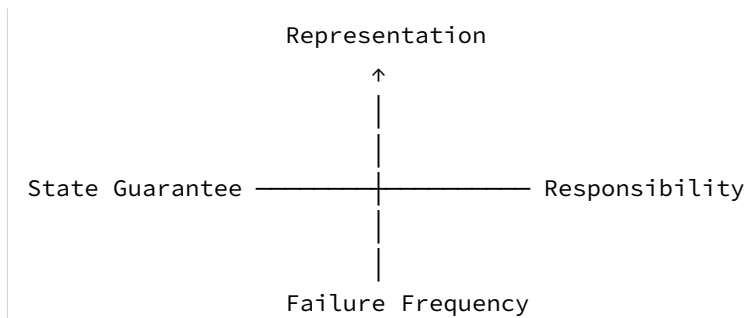
通常：one input object

而不是：entire process

这就是 error containment。

45. Failure Model 的二维底图

至少先建立：



实际工程中还需要再加入：

Recovery Boundary
Failure Domain

因此它不是简单的一维：expected vs exception 问题。

七、接口合同与反例

46. API Failure Contract

一个 API 的失败合同先回答输入和成功，再分别描述失败分支、状态与资源、恢复权限。相同合同不应在 FM-0、FM-4 和 FM-9 各维护一份模板。

统一填写入口为 [FM-9 §26](#)；八项公共审查问题见系列阅读约定。以下旧细项定位保留为该合同的历史入口。

47. API Review Template

对接口：

```
R operation(A input);
```

先完成公共 C1–C8：输入与成功、失败集与通道、状态与提交、所有权与清理、组合前提、恢复与影响范围、重试与控制状态、验证与限制。

本章的作用是把这些问题连成模型，不重复 FM-9 的填写模板。尤其不要把发生频率和严重性混在一起，也不要终止写成 strong/basic 之后的保证等级。旧审查细项锚点仍定位到本节，语义由统一合同承接。

48. Decision Baseline

以下不是绝对规则，而是工程默认值。

场景	默认考虑
正常 absence	std::optional<T>
可预期且需要原因的失败	std::expected<T, E>
OS / system API 错误	std::error_code 或封装后的领域错误
稀有、需要跨多层传播的失败	exception
外部不可信输入	validate + structured failure
调用者违反前置条件	contract/assert/fail-fast
内部不变量损坏	fail-fast / terminate
高频普通分支	value-based result
ABI / thread / process 边界	捕获、转换、隔离
无法确定程序状态	不应假装恢复

49. Anti-Patterns

49.1 Catch and Continue

```
try {
    mutate_state();
} catch (...) {
    log();
}

continue_running();
```

如果不知道: state after failure
这不是 recovery。

49.2 Catch All Too Low

```
std::string read_file(...) {
    try {
        ...
    } catch (...) {
        return {};
    }
}
```

这把: empty file
和: file read failed
混在一起。
同时丢失恢复上下文。

49.3 Assert External Input

```
错误:
assert(packet.size() >= header_size);
```

如果 `packet` 来自网络，这是：untrusted runtime input
应该验证并报告 `failure`。

49.4 Expected for Impossible Internal State

如果某件事根据程序 invariant: must never happen
把它变成：

```
std::expected<T, ImpossibleError>
```

然后一路传播，可能只是在隐藏 bug。

49.5 Low-Level Retry Without Context

底层擅自：retry forever

可能破坏：

```
deadline  
shutdown  
idempotency  
backpressure  
retry budget
```

49.6 Translate Every Layer

不要构造：

```
SocketError  
  → ClientError  
  → StorageError  
  → RepositoryError  
  → ServiceError  
  → ApplicationError
```

除非每次 translation 都真的产生新的抽象语义。

49.7 Error String as Error Model

不要把：

```
std::string error;
```

当成主要机器可判定错误模型。

字符串适合：

```
diagnostics  
logging  
human context
```

结构化类型适合：

```
control flow  
policy  
recovery
```

八、系统状态与副作用

50. Failure Model 与 Logging

日志 (logging) 服务诊断，不是错误返回通道。下层保存 / 补充上下文，由真正决定恢复或终止的边界记录最终诊断，可避免一个失败沿五层传播就变成五条 ERROR。

这是一种可裁剪的工程策略，不是禁止各层产生有区分的 trace；报告自身的分配、格式化和 I/O 失败也需要后备路径。具体边界见 FM-8 §22，项目政策见 FM-9 §18-20。

51. Failure Model 与 Observability

一个失败可能需要同时产生：

```
structured error
metric
trace event
log
```

但这些职责不同：

```
error object
    → program control flow

metric
    → aggregated operational state

trace
    → request causality

log
    → human diagnosis
```

不要让 logging string 取代结构化错误。

52. Failure Model 与 Context

错误向上传播时可能需要增加 context：ENOENT

单独看价值有限。

更有诊断价值的是：

```
failed to load project configuration
path=/etc/app/project.toml
cause=no such file or directory
```

但应区分：machine-readable category

和：human-readable context

理想错误模型通常保留两者。

53. Failure Model 与 Shutdown

shutdown 本身应该视为一种特殊控制状态。

例如: `network read interrupted because system is shutting down`

不一定应该被统计为: `network failure`

因此成熟系统通常需要区分:

```
operation failed
operation cancelled
operation timed out
system shutting down
```

否则 observability 会产生错误信号。

54. Failure Model 与 Cancellation

Cancellation 不应该自动等同于 error。

例如:

```
user cancelled
deadline expired
shutdown requested
superseded operation
```

这些可能属于: `control outcome`

而不是: `component defect`

因此 cancellation 应在 Failure Model 中拥有明确语义。

55. Failure Model 与 Idempotency

重试前先确认操作是否幂等、是否明确未提交,或是否由接收方提供了可依赖的去重 / 查询协议;再检查截止期、预算、关闭状态与恢复权限。只看 GET、错误名称或“有 request ID”都不足以证明再次执行安全。

例如付款在响应丢失时可能已经提交,再发一次可能重复扣款。恢复协议及其边界统一见 [FM-8 §17-19](#)。

56. Failure Model 与 Partial Commit

提交事实与调用方观察必须分开:写文件、发布消息或远端请求可能已经生效,返回 / 确认却失败。调用方此时只能报告完成状态未知,不能把“我没看到成功”改写为“没有副作用”。

本地状态转换的推理主讲在 [FM-4 §7](#),远端完成不确定性在 [FM-8 §17](#)。两者都要求明示观察范围,不把内存提交、持久化和分布式确认视为同一保证。

57. Failure Model 与 Ownership

一个 operation 失败时必须知道：谁还拥有资源？

例如：

```
send(std::unique_ptr<Job> job);
```

失败以后：

```
job ownership transferred?  
or retained by caller?
```

必须明确。

这也是为什么现代 C++ 的 ownership 类型对 Failure Model 很重要。

58. Failure Model 与 Move

移动操作尤其需要明确：

```
failure before ownership transfer  
failure during ownership transfer  
source state after failure
```

这直接影响：

```
strong guarantee  
rollback  
container relocation
```

后续在 FM-5 专门展开。

九、项目交接与回查

59. 设计层次

推荐将大型系统 failure strategy 分成三层。

Layer 1 — Local API Contract

每个函数描述：local failure semantics

Layer 2 — Subsystem Failure Policy

例如：

```
storage subsystem  
network subsystem  
parser subsystem  
worker subsystem
```

定义：

- error taxonomy
- retry
- fallback
- failure boundary

Layer 3 — Application Failure Policy

统一决定：

- logging
- metrics
- shutdown
- restart
- process termination
- operator alerts

这样避免每个函数各自发明策略。

60. 项目级 Failure Profile

项目 Failure Profile 用于选择本项目允许的通道、保证、边界和恢复政策，不是新的 C++ 语言规则。它应汇总 API 的合同，而不是替代每个 API 的状态分析。

唯一填写入口放在 FM-9 的 API 与子系统模板。本系列的完成或接受，不代表任何具体项目已经采纳该模板。

61. Failure Model 的核心不变量

以下原则建议作为长期工程基线。

FM-I1

Failure 与 exception 必须分离讨论。

FM-I2

Normal absence 不应伪装成 exceptional failure。

FM-I3

External invalid input 不应使用 assertion 代替 validation。

FM-I4

Programmer error 不应轻易降级成普通业务错误继续运行。

FM-I5

Detection point 不等于 recovery point。

FM-I6

Recovery policy 必须由拥有足够上下文的层决定。

FM-I7

Handling 不等于 recovery。

FM-I8

Error representation 与 state guarantee 是两个独立维度。

FM-I9

一个错误机制的好坏必须结合 failure frequency 判断。

FM-I10

Failure 必须被限制在明确的 failure domain。

FM-I11

跨 thread / ABI / process boundary 时必须明确 error conversion。

FM-I12

如果无法证明系统已经恢复到定义良好的状态，就不要假装 recovery 已经完成。

FM-I13

Retry 是业务恢复策略，不是底层错误处理默认动作。

FM-I14

Error translation 只应发生在抽象语义真正变化的地方。

FM-I15

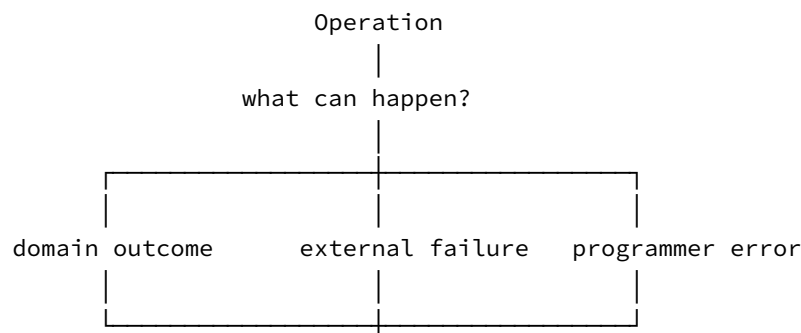
Resource ownership 必须在所有 failure path 上保持明确。

62. 最终心智模型

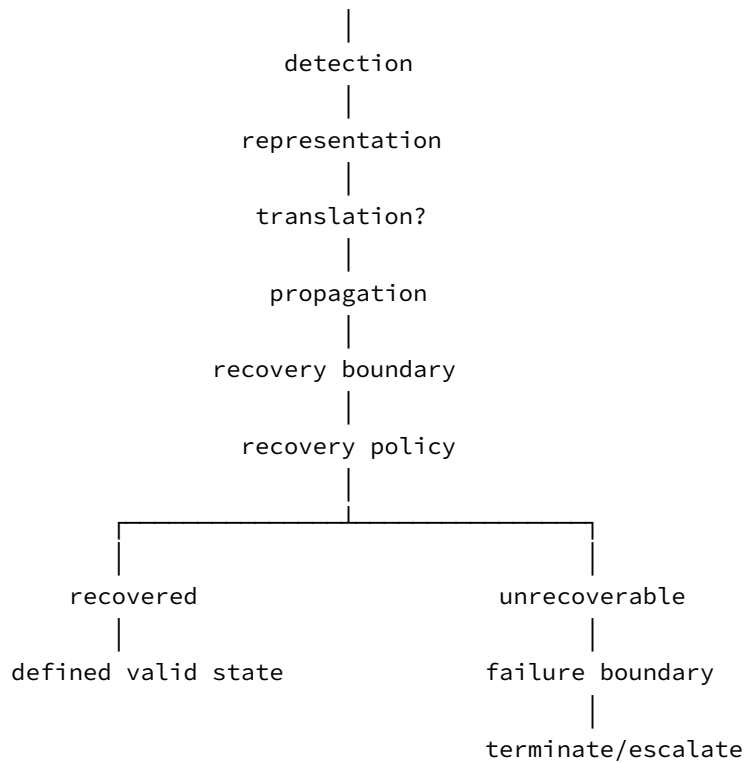
分析任何 operation 时，不要先想：

throw?
expected?

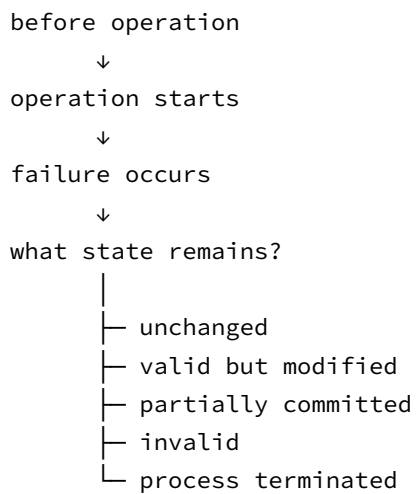
先建立：



FM-0 · 62. 最终心智模型 · continued



同时沿另一条轴检查：



图中的 process terminated 是终局处置，不是存活对象的状态保证等级。

只有这两个模型都清楚之后，才讨论：

```

optional
expected
error_code
exception
assert
terminate
  
```

63. FM-0 Review Checklist

先完成公共 C1–C8，本章另外检查：

- 是否把领域结果、故障原因、错误表示和对外失败分开？
- detection、recovery boundary 与 failure domain 是否各有职责，而非都写“上层处理”？
- 同一个错误能否沿表示 / 传播链回到它影响的状态？
- “已恢复”是否有定义良好的后置条件，而不只是 catch 或日志记录？
- 模型中的未知、部分提交和终止是否仍显式保留？

64. FM-0 压缩版

最终可以把整个 FM-0 压成六个问题：

1. What happened?
发生了什么？
2. Whose responsibility is it?
属于正常结果、环境失败还是程序缺陷？
3. What remains valid?
失败以后还有哪些状态保证？
4. Who knows what to do?
哪一层拥有恢复所需上下文？
5. How far should the failure spread?
failure domain 到哪里？
6. How should it travel?
最后选择 expected / exception / error_code；终止作为独立处置说明。

如果这六个问题能够准确回答，一个 C++ Failure Model 通常已经具备正确的骨架。

65. 后续学习位置

FM-0 只建立统一模型。

后续章节分别解决具体机制：

FM-1
Contracts / Preconditions / Assertions / UB

FM-2
Value-based Failure
bool / optional / expected / error types

FM-3

FM-0 · 65. 后续学习位置 · continued

C++ Exception Semantics

throw / catch / stack unwinding

FM-4

RAII & Exception Safety

basic / strong / transactional design

FM-5

noexcept / Move / Copy

container guarantees / move_if_noexcept

FM-6

Construction / Destruction / Allocation Failure

FM-7

error_code / system_error / OS Failure

FM-8

Thread / Coroutine / ABI / RPC Failure Boundaries

FM-9

Project-Level Failure Profile

FM-0 的职责到这里结束：

先建立正确的 **failure vocabulary**、**responsibility model**、**state model**、**recovery boundary** 和 **failure domain**，再选择具体语言机制。

这套顺序将作为后续现代 C++ Failure Model 的统一分析框架。

FM-1 · 契约与可信边界

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-0](#) · [下一章：FM-2](#) · [术语与审查约定](#)

阅读入口

明确前置条件、成功 / 失败后置条件、不变量、**validation** 与 **assertion** 的责任。UB 和终止不能混成一种错误通道。

主阅读线。§1-8 → §12-18 → §19-26 → §33-40；具体反例按需回查。

失败契约。从未经验证输入进入可信表示后，仍须维护生命周期、别名与同步条件。失败可以是合法运行结果；违约的具体后果要逐个接口判断。

证据边界。T01/T02、T04/T05 定向检查异常构造、枚举与 **unreachable**；不证明所有前置条件或 UB 都会被诊断。无 **fm-test** 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、合同与责任
- 二、验证边界与断言
- 三、检查访问与未定义行为
- 四、终止与不可达承诺
- 五、可信表示与类型设计
- 六、反例与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、合同与责任

0. 文档定位

本文解决现代 C++ Failure Model 中一个基础问题：

什么情况应该被建模为“可恢复失败”，什么情况应该被视为“程序违反了自身契约”？

如果这个边界不清楚，工程中很容易出现以下问题：

- 用 `assert` 检查不可信输入；
- 把程序 bug 包装成 `std::expected` 一路传播；
- 把 UB 当作一种“性能更高的错误处理”；
- `release` 构建中因为 `assert` 消失而暴露安全问题；
- 在 `trusted core` 中重复进行大量无意义检查；
- 在 `validation boundary` 之后仍然传播“半可信”数据。

FM-1 的目标是建立：

```
external uncertainty
  ↓
validation
  ↓
trusted representation
  ↓
contract-based core
```

并明确区分：

```
runtime failure
contract violation
undefined behavior
fail-fast
```

1. Contract 是什么

一个函数的完整接口不只是：

```
R f(A a);
```

真正的 `contract` 至少包含：

```
preconditions
postconditions
invariants
failure semantics
```

2. Preconditions

Precondition 表示：

调用者在调用函数之前必须保证的条件。

例如：

```
int front(std::span<const int> values) {
    return values.front();
}
```

一个关键 **precondition** 是: `!values.empty()`

如果调用者违反这个条件, 含义不是: `front()` 正常尝试工作但失败

而是: 调用本身违反 **API contract**

因此: **recoverable runtime failure**

和: **precondition violation**

必须严格区分。

3. Postconditions

后置条件 (**postcondition**) 描述指定完成分支之后应成立的条件。**正常返回不一定表示业务成功**: 返回错误值也是正常返回, 必须区分成功后置条件与失败后置条件。

例如:

```
void sort_values(std::span<int> values);
```

成功返回后, `values` 应满足约定排序。又例如:

```
File open_file(...);
```

若该接口约定“正常返回即成功”, 返回的 `File` 应拥有有效资源; 若采用 **expected**, 则成功与错误分支各有自己的后置条件。实现没有满足适用分支的合同才属于被调方缺陷, 不能把正常返回的错误值误判为违约。

4. Invariants

Invariant 表示:

一个类型或组件在所有公开可观察的合法状态中都必须成立的条件。

例如:

```
class Buffer {
private:
    std::byte* data_;
    std::size_t size_;
};
```

可能定义:

```
size_ == 0
⇔
data_ == nullptr
```

或者:

```

size_ > 0
    ⇒
data_ != nullptr

```

对于:

```

class RingBuffer

```

可能有:

```

size_ <= capacity_

head_ < capacity_

tail_ < capacity_

```

Invariant 是: valid object state 的定义。

5. Contract 的责任方向

对于:

```

R f(A a);

```

责任可以理解为:

```

caller
  |
  | establishes preconditions
  ▼
callee
  |
  | performs operation
  ▼
postconditions
  |
  ▼
object/component invariants remain valid

```

因此:

```

Precondition
    → caller responsibility

Postcondition
    → callee responsibility

Invariant
    → type/component responsibility

```

6. Runtime Failure 与 Contract Violation

这是 FM-1 最重要的分界。

Runtime Failure

环境本来就可能不满足要求。

例如：

```
network packet truncated
file missing
permission denied
invalid configuration
remote timeout
malformed user input
```

程序必须正常面对这些情况。

典型处理：

```
validate
→ structured failure
→ propagate / recover
```

Contract Violation

根据 API 或程序设计：这个条件本来必须成立

但调用者或内部实现破坏了它。

例如：

```
invalid internal index
illegal state transition
dangling reference
double ownership
impossible enum/state combination
```

这属于：programmer error

而不是普通业务失败。违约后是否 UB、显式终止或定义良好的错误报告，由具体合同与语言规则决定；“契约违反”本身不是一个统一的运行时机制。

7. 判断标准

不要只看错误表面形式。

例如：index out of range

可能属于完全不同的失败模型。

外部输入

```
index = request.index;
```

如果来自 HTTP、文件、网络或数据库：runtime data

必须先判断其可信度。

如果可能非法: `validation`

内部已验证状态

如果:

```
index = validated_table[id];
```

并且程序 `invariant` 已保证: `index < size`

那么越界说明: `program bug`

8. 核心问题

判断一个条件属于 `precondition` 还是 `runtime validation` 时, 优先问:

谁有责任保证这个条件成立?

如果调用者根据 `API contract` 必须保证: `precondition`

如果外部环境天然可能违反: `runtime validation`

二、验证边界与断言

9. Validation

`Validation` 用于处理: `untrusted or runtime-controlled data`

包括:

```
network packets
files
user input
RPC requests
database records
configuration
external service responses
serialized state
```

典型流程:

```
untrusted data
  ↓
validation
  ↓
valid internal representation
```

失败应该产生: `structured recoverable failure`

例如:

```
std::expected<Frame, ParseError>
parse_frame(std::span<const std::byte> bytes);
```

10. Assertion

Assertion 的用途不同：

检查程序自己已经推理为“必须成立”的条件。

例如：

```
assert(index < messages.size());  
  
return messages[index];
```

含义是：

如果这里失败，
说明程序内部推理或 `invariant` 被破坏。

不是：用户给了坏数据，请优雅处理。

11. Validation 与 Assertion 的工程区分

可以压缩为：

```
validation  
    protects the program from external uncertainty  
  
assertion  
    checks the program against its own assumptions
```

这是本节最重要的工程规则之一。

12. assert

C++：

```
#include <cassert>  
  
assert(condition);
```

主要用于开发阶段检查内部条件。

如果 `assertion enabled` 且条件为 `false`：

```
diagnostic  
→ abort
```

具体输出形式由实现决定。

13. NDEBUG

是否启用标准断言取决于包含 `<cassert>` 时的 `NDEBUG`，而不是语言认识某个 `Debug` / `Release` 构建名称：

```
#define NDEBUG
```

在禁用断言的配置下：

```
assert(expr);
```

其表达式不会求值。因此下例不能承担必要输入验证：

```
assert(validate_input());
```

Release 常定义 NDEBUG，但这是构建配置约定；Debug 也可能禁用断言。需要一直执行的验证必须写在断言之外。

14. Assertion 不得承载必要副作用

错误：

```
assert(++index < size);
```

debug: index incremented

release: index unchanged

导致程序语义依赖 build mode。

同样：

```
assert(initialize_resource());
```

也是危险设计。

原则：

`assert` 中的表达式应当主要用于观察和验证，而不是承担程序正确运行所依赖的副作用。

15. assert 不适合处理外部输入

错误：

```
void decode_packet(std::span<const std::byte> packet) {  
    assert(packet.size() >= kHeaderSize);  
  
    ...  
}
```

如果 packet 来自网络: packet too short

属于: runtime input failure

而不是: programmer bug

正确模型：

```
std::expected<Packet, ParseError>
decode_packet(std::span<const std::byte> packet) {
    if (packet.size() < kHeaderSize) {
        return std::unexpected(ParseError::truncated);
    }

    ...
}
```

16. Validation Boundary

成熟系统应该明确：

```
untrusted world
    ↓
validation boundary
    ↓
trusted domain object
    ↓
trusted core
```

例如：

```
raw bytes
    ↓
parse + validate
    ↓
Frame
```

成功得到：

```
Frame frame;
```

最好已经意味着：

```
length valid
version valid
fields consistent
checksum validated
required ranges valid
```

17. Validate Once, Trust Afterwards

一种高质量设计模式：

```
external representation
    ↓
checked boundary
    ↓
validated type
    ↓
trusted core
```

不要让：

```
RawPacket
```

贯穿整个程序，并在每一层重复：

```
if (!packet.valid()) {
    ...
}
```

更好的设计：

```
class Frame {
public:
    static std::expected<Frame, ParseError>
    parse(std::span<const std::byte> bytes);

private:
    Frame(...);
};
```

成功构造 `Frame` 本身就表达： `Frame invariant holds`

该工程模式的前提是校验结果在整个使用期内持续有效。若 `Frame` 借用外部内存，必须保证生命周期、别名修改和线程同步不会使已验证条件失效；包装成 `validated` 类型本身不能替代这些责任。发生相关变更后应重新建立不变量或重新验证。

18. Checked Boundary + Trusted Core

高性能系统中非常常见：

```
external bytes
    ↓
checked parser
    ↓
validated representation
    ↓
unchecked hot path
```

例如：

```
std::expected<Message, ParseError>
parse_message(std::span<const std::byte> bytes);
```

负责：

```
length validation
version validation
field range checks
header validation
```

而内部：

```
decode(const Message& message);
```

直接依赖: Message invariant

这可以同时获得:

```
correctness
performance
clear ownership of validation
```

三、检查访问与未定义行为

19. operator[] 与 at()

这是 contract-based API 与 checked API 的经典对比。

```
std::vector<int> values{1, 2, 3};
```

operator[]

```
values[index];
```

调用者负责保证: `index < values.size()`

在 C++23 语义基线下, 如果越界: undefined behavior

它不是一个正常错误返回通道。

at()

```
values.at(index);
```

由容器主动检查: `index < size()`

越界:

```
throw std::out_of_range
```

因此:

```
operator[]
    caller establishes precondition

at()
    callee validates index
```

20. Checked 与 Unchecked API 都有合理用途

如果:

```
if (index >= values.size()) {
    return std::unexpected(Error::invalid_index);
}

auto& value = values[index];
```

那么检查已经发生。

这里：

```
operator[]
```

是合理的 `trusted-core` 操作。

如果：

```
return values[request.index];
```

而 `index` 直接来自外部，则不合理。

因此问题不是：`at()` 永远比 `[]` 好

而是：`check` 在哪个 `boundary` 完成？

21. Undefined Behavior

UB 不是错误处理机制。

它表示：

程序已经执行了 C++ 标准不再规定语义的操作。

例如：

```
std::vector<int> values{1, 2, 3};
```

```
int x = values[100];
```

这里不是：C++ 返回一个随机值

而是：language guarantees have ended

22. UB 可能产生什么结果

UB 以后可能：

- 看起来正常
- crash
- 读垃圾值
- 写坏内存
- 删除某些代码路径
- 产生违反直觉的优化结果

不能依赖其中任何一种。

23. UB 与 Runtime Error 的根本区别

定义良好的失败报告仍处于 C++ 语义规则之内。例如下面是抛出语句片段，需要 `<stdexcept>` 和 外围函数：

```
throw std::out_of_range{"index out of range"};
```

`std::out_of_range` 需要错误描述参数, 不能写成 `std::out_of_range{}`。这里“运行时失败”指运行期间被显式报告的失败, 不是说该类型继承自 `std::runtime_error`; 它实际属于 `std::logic_error` 分支。N4950: [out.of.range](#)

调用者可依据明确的抛出、捕获和状态契约继续推理。UB 则没有这种可移植的恢复保证。

完整正例和原式的预期编译失败反例见 T01 / T02。

24. UB 会影响优化推理

例如:

```
int f(int* p) {
    int x = *p;

    if (p == nullptr) {
        return 0;
    }

    return x;
}
```

`*p` 已经要求:

`p != nullptr`

所以在任何定义良好的执行中: `p cannot be null`

编译器可以据此优化后面的:

```
if (p == nullptr)
```

这说明:

UB 不只是“运行时出了问题”, 它还改变了编译器对程序合法路径的推理。

25. assert 与 UB

在断言启用且条件求值本身有定义时:

```
assert(index < size);
```

假条件走诊断与终止路径。若断言被禁用, 条件不求值, 也没有这道保护; 随后执行:

```
array[index]
```

仍须满足实际访问的前置条件。断言能辅助在 UB 前发现某些违约, 但不使非法访问变成错误值, 也不保证所有缺陷都被检查。

26. Assertion 不会改变 API Contract

例如:

```
T& get(std::size_t index) {  
    assert(index < size_);  
    return data_[index];  
}
```

真实 contract 仍然是：

```
Precondition:  
    index < size_
```

assert 只是：

debug-time checking of that contract

它不是：runtime validation API

因为 release 构建可能没有这个检查。

27. 常见 UB 来源

Memory

```
out-of-bounds access  
use-after-free  
dangling pointer/reference  
invalid pointer arithmetic  
misaligned access
```

Lifetime

```
use outside object lifetime  
use destroyed object  
incorrect manual lifetime management
```

Arithmetic

例如：

```
int x = INT_MAX;  
++x;
```

signed integer overflow: UB

而无符号整数: modulo arithmetic

不是 UB。

Object Model

可能包括：

```
invalid type punning  
alignment violations  
incorrect object representation assumptions
```

Concurrency

未同步的数据竞争: data race

通常就是：UB

不能把它理解为：偶尔读取旧数据

28. 为什么 C++ 允许 Contract-Based Unchecked Operations

Unchecked 操作允许调用者建立前提后由被调方依赖它。例如：

```
values[index];
```

不规定必须执行如下检查：

```
if (index >= values.size())
```

这给实现和调用者控制成本的空间，但不证明 checked API 必然更慢；优化器可能消除冗余检查，性能仍须测量。本章关注的是证明与验证责任，不能以“零开销”口号允许未经验证的数据进入前置条件接口。

四、终止与不可达承诺

29. Fail-Fast

当内部 invariant 被破坏时，继续运行有时比终止更危险。

例如：

```
ownership corrupted  
state machine impossible state  
critical accounting invariant broken  
heap corruption suspected
```

此时合理策略可能是：

```
detect  
→ record diagnostics  
→ terminate affected failure domain
```

这叫：fail-fast

30. Fail-Fast 不等于 UB

应优先区分：explicit termination

和：undefined behavior

一个系统发现 impossible state 后：

```
std::terminate();
```

比故意继续执行直到 UB 更有定义、更容易诊断。

31. Debug Assertion 与 Production Check

标准：

```
assert(condition);
```

可能在 release 消失。

但有些 invariant：即使 production 也必须验证

例如继续运行可能造成：

```
persistent data corruption
security violation
irreversible external side effect
```

那么项目通常需要：always-on fatal check

概念上：

```
CHECK(condition);
```

其行为：

```
debug:
    check

release:
    still check

failure:
    diagnostic + terminate
```

CHECK 不是标准 C++ API，通常属于项目基础设施。

32. Assertion 与 Production Invariant Check

推荐区分：

```
assert
    development correctness aid

always-on fatal check
    production invariant enforcement
```

不要因为：这是内部 invariant

就自动认为：release 可以不检查

是否需要 always-on 检查取决于 violation 的后果。

33. `std::terminate`

`std::terminate()` 表示：

program cannot continue through normal C++ execution

它具有定义明确的终止语义。

典型触发包括：

```
exception escapes noexcept function
uncaught exception escapes std::thread entry
explicit fail-fast decision
```

它属于：terminal behavior

而不是普通错误 transport。

34. `std::unreachable` — C++23

`std::unreachable()` 承诺合法执行不会到达此处；到达它是 UB，不是受控拒绝。N4950: [utility.unreachable](#)

以下是带前置条件的函数片段：

```
#include <utility>

enum class State { idle, running };

// Preconditions: state == State::idle || state == State::running.
int value(State state) {
    switch (state) {
        case State::idle:    return 0;
        case State::running: return 1;
    }
    std::unreachable();
}
```

该 `scoped enum` 的固定底层类型是 `int`；`static_cast<State>(2)` 本身可以有定义，却不是两个命名值之一。若不能在调用前证明前置条件，就应在边界验证，或将最后一行替换为明确的拒绝 / 终止路径，而不是保留不可达假设。N4950: [expr.static.cast/10](#)

T04 / T05 分别检查未命名值与隔离的 UBSan 反例；sanitizer 的一次诊断不构成 UB 的可移植行为保证。

35. `std::unreachable` 不是 Fail-Fast

它不是：

```
abort
terminate
panic
throw
```

它是在向编译器声明: `this control-flow path is impossible`

如果真实执行到:

```
std::unreachable();
```

行为是: `undefined behavior`

因此:

`std::unreachable` 是语义/优化承诺, 不是运行时保护机制。

36. 三种机制对比

机制	条件违反时
<code>assert(false)</code>	<code>assert enabled</code> 时终止; 可能被禁用
<code>std::terminate()</code>	明确定义为程序终止
<code>std::unreachable()</code>	执行到即 UB

不要互相替代。

37. `std::unreachable` 的使用原则

只有当你可以证明: 合法程序状态下绝不可能到达才考虑使用。

不要习惯性写:

```
default:
    std::unreachable();
```

尤其当:

```
enum may receive corrupted value
ABI compatibility exists
serialized state may be stale
external input may participate
```

时更要谨慎。

在很多高可靠系统中: `diagnostic fail-fast`

比极小的潜在优化价值更重要。

五、可信表示与类型设计

38. Internal 不等于 Trusted

这一点非常重要。

数据来自：

```
database
cache
disk
old process version
IPC
shared memory
another service
```

即使这些组件属于“自己的系统”，数据也未必可信。

例如：数据库字段由旧版本写入

当前程序仍然应该把它视作：runtime externalized state

而不是：guaranteed internal invariant

因此：

“来自内部系统”不等于“满足当前进程 invariant”。

39. 信任应该建立在 Boundary 上

推荐：

```
raw persisted/external state
    ↓
validation
    ↓
current-version validated representation
    ↓
trusted core
```

而不是根据：数据是不是我们自己写的

决定是否信任。

40. Type as Invariant

最好的 invariant 往往不是：

```
assert(valid());
```

而是：非法状态根本无法通过类型表达

例如不要设计：

```
struct Connection {  
    Socket socket;  
    bool connected;  
};
```

因为存在：

```
socket invalid  
connected == true
```

这种矛盾组合。

可以考虑：

```
DisconnectedConnection  
ConnectedConnection
```

让状态转换通过类型发生。

41. Invalid States Should Be Hard to Represent

现代 C++ 类型设计应尽量：encode invariants into construction and type structure

利用：

```
private constructors  
factory functions  
strong types  
enum class  
variant  
RAII  
ownership types  
validated domain types
```

将错误从：runtime checking problem

提升为：construction/type-system problem

42. 优先级

如果一个非法状态能够：编译期消除

优先编译期。

否则：构造阶段消除

再否则：validation boundary 消除

最后才是：trusted core assertion

可以理解成：

```

compile-time
  ↓
construction-time
  ↓
boundary validation
  ↓
internal invariant check

```

越早消除越好。

43. Engineering Decision Model

面对：

```

if (!condition) {
  ...
}

```

首先判断 condition 为什么可能失败。

Case A — 外部世界可能不满足

```

validation
→ recoverable failure

```

Case B — API 明确允许任意输入

```

checked API
→ recoverable failure

```

Case C — 调用者必须保证

```

precondition
→ contract

```

Case D — 内部逻辑保证成立

```

invariant
→ assertion / fatal check

```

Case E — 无法恢复且状态可信度已经丢失

fail-fast

44. API Design Pattern

推荐形成两层 API：

```

checked public/external boundary
  ↓
validated representation
  ↓
fast internal API with clear preconditions

```

例如：

```
std::expected<Message, ParseError>
parse_message(std::span<const std::byte> bytes);
```

然后：

```
void decode(const Message& message);
```

而不是：

```
void decode(std::span<const std::byte> bytes);
```

并在所有深层函数里重复检查原始字节。

六、反例与回查

45. Anti-Patterns

45.1 Assert Untrusted Input

错误：

```
assert(packet.size() >= 32);
```

如果 packet 来自外部。

45.2 Depend on Assert Side Effects

错误：

```
assert(initialize());
```

45.3 Convert Every Programmer Bug to expected

错误倾向：

```
std::expected<T, InternalImpossibleState>
```

如果这个状态按设计根本不应该发生。

结果可能只是：bug 被隐藏并传播

45.4 Treat UB as Fast Error Handling

错误思维：我们不检查，错了反正 UB，这样最快。

UB 不是 failure policy。

合理模型应该是：

```
validated boundary
+
trusted precondition
```

而不是：unchecked untrusted input

45.5 Revalidate Everywhere

如果：ValidFrame

已经保证 invariant,

不要所有内部函数都再次验证：

```
header valid?
length valid?
version valid?
```

这会：

```
增加复杂性
模糊 trust boundary
增加重复成本
削弱类型语义
```

45.6 Blind std::unreachable

不要用：

```
std::unreachable();
```

掩盖：其实没有证明 impossible

46. Review Checklist

先用公共 C1-C8 合同检查完整操作，再用以下问题回查本章机制。

设计或 Review API 时：

- ☐ Preconditions 是否明确？
- ☐ Preconditions 由谁负责建立？
- ☐ Postconditions 是否明确？
- ☐ 类型 invariants 是否明确？
- ☐ 外部数据是否经过 validation boundary？
- ☐ Internal data 是否真的值得信任？
- ☐ 是否错误使用 assert 处理外部 failure？
- ☐ assert 是否包含必要副作用？
- ☐ release 构建是否会失去必要检查？
- ☐ 是否需要 production fatal check？
- ☐ UB 是否可能由未验证输入触发？
- ☐ checked API 与 trusted API 是否职责明确？
- ☐ validated representation 是否尽早建立？
- ☐ 是否存在可以由类型系统消除的非法状态？
- ☐ std::unreachable 是否有严格证明？
- ☐ contract violation 是否被错误地当作业务失败传播？

47. FM-1 核心不变量

FM1-I1

External uncertainty 必须通过 validation 进入系统。

FM1-I2

Assertion 用来检查内部推理，而不是验证不可信输入。

FM1-I3

Precondition violation 与普通 runtime failure 是不同类别。

FM1-I4

assert 由 NDEBUG 控制，是可禁用的合同检查，不是可靠的常开输入验证。

FM1-I5

不得让程序正确性依赖 `assert` 中的副作用。

FM1-I6

UB 不是 error transport, 也不存在通用 recovery contract。

FM1-I7

应在 UB 发生之前检测重要 contract violation。

FM1-I8

`std::unreachable` 是“不可到达”的语义承诺，不是 fail-fast API。

FM1-I9

数据是否可信由 validation boundary 决定，不由“是不是自己的系统产生”决定。

FM1-I10

Checked boundary 和 trusted core 应明确分层。

FM1-I11

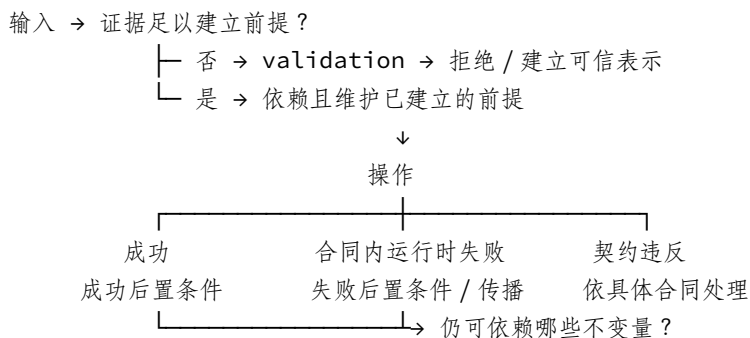
高价值 invariant 应尽可能编码进类型和构造过程。

FM1-I12

如果继续运行已经无法保证系统语义，应显式 fail-fast，而不是进入 UB 后期待恢复。

48. 最终心智模型

从输入到操作结果，应始终区分合法运行时失败与契约违反：



UB 不是图中的恢复分支。即使某次系统调用失败、分配失败或抛出异常，也不意味着调用方前提被破坏；反过来，某个前提被破坏后是否 UB、诊断或定义良好的拒绝，取决于具体规则。目标是在非法操作发生前建立必要条件，而不是为 UB 设计通用 catch。

49. FM-1 最终压缩

FM-1 可以压成四句话：

外部不确定性用 **validation** 处理。

内部必然条件用 **contract/invariant** 描述。

违反 **contract** 是程序缺陷，不应伪装成普通业务失败。

UB 不是失败机制；高质量系统应该在 UB 之前建立 **validation**、**assertion** 或 **fail-fast boundary**。

这构成 FM-2 讨论 optional、expected、error type 和 value-based failure 的前提。

FM-2 · 值通道与错误类型

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-1](#) · [下一章：FM-3](#) · [术语与审查约定](#)

阅读入口

主讲合法 `absence`、失败值、错误对象和组合条件。是否恢复由策略决定；是否 `strong` 或 `no-throw` 要回到状态与完整表达式。

主阅读线。 §1-8 → §10-16 → §19-21；先理解调用者要做什么决策，再选类型。

失败契约。 错误返回同样是正常返回分支；记录 T/E 构造能否抛出、错误时受保护状态是否变化，以及按值资源能否返还。

证据边界。 正文 T18/T19 分别检查纯内存管道与所有权返还；T06-T10/T16 在附录。它们不验证真实 I/O 或所有 `monadic` 重载。无 `fm-test` 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、结果空间与表示
- 二、错误类型与传播
- 三、组合条件与状态责任
- 四、成本、反例与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、结果空间与表示

0. 文档定位

本章主讲如何把操作结果设计成值：成功值、合法 `absence` 和结构化错误分别携带调用者需要的信息。值通道并不要求每种失败都能就地恢复；它也可以把事实交给更高层决定拒绝或终止。

核心形式是 `SuccessValue` | `FailureValue`，主要通过普通 `return path` 传播。C++23 的主要工具之一是：

```
std::expected<T, E>
```

`bool`、`sentinel`、`enum/status`、`optional` 与 `error_code` 仍有各自用途。选型依据是结果语义、调用者决策与边界，不是“越新的类型越好”。

1. bool

接口片段：

```
bool try_push(Item item);
```

只有该接口明确约定时，`true` / `false` 才分别表示入队成功 / 拒绝；失败后的 `item` 所有权还要单独说明。下列两个 `bool` 接口表达的又是不同结果：

```
bool try_lock() noexcept;
bool contains(Key key) const;
```

`contains` 的 `false` 通常是合法的否定答案，不是一次查询失败；`try_lock` 未取得锁也可以是正常竞争结果。返回类型本身不定义 *failure taxonomy*。

当调用者只需要二分决策时，`bool` 足够；如果还需要区分不存在、权限、超时或资源耗尽，则：

```
bool open();
```

会丢掉必要语义，应选择能携带原因的接口。

2. Sentinel

传统接口经常：

```
std::size_t find(...);
```

约定：

```
normal index → success
npos         → not found
```

`sentinel` 的根本问题是：

```
success domain
和
failure domain
```

共享同一个值空间

调用者必须额外知道：哪些值不是普通值

现代 API 如果能够表达为：

```
std::optional<std::size_t>
```

通常具有更明确的类型语义。

但已有标准接口中的 `sentinel`，例如：

```
std::string::npos
```

仍然是完全合法并广泛使用的设计。

3. `std::optional<T>`

`std::optional<T>` 表达:

`T | absence`

例如:

```
std::optional<User> find_user(UserId id);
```

表示:

```
User exists  
or  
User does not exist
```

它特别适合:

```
lookup miss  
optional configuration  
optional field  
search result  
cache miss
```

核心规则

`optional` 应表达 `absence`，而不是把多个 `failure reason` 压缩成“没有值”。

例如:

```
std::optional<Config> load_config();
```

如果空值可能代表:

```
file missing  
permission denied  
parse failure  
I/O failure
```

那么信息模型通常过弱。

4. `std::expected<T, E>`

C++23:

```
std::expected<T, E>
```

直接表示: `T | E`

即:

```

success value
or
structured failure

```

`std::expected` 本身始终处于 `value` 或 `error` 两种状态之一，不存在第三种 `valueless` 状态；C++23 同时提供 `and_then`、`transform`、`or_else`、`transform_error` 等 monadic operations。

例如：

```

enum class ParseError {
    truncated,
    invalid_version,
    invalid_checksum,
};

std::expected<Frame, ParseError>
parse_frame(std::span<const std::byte> input);

```

接口直接表达：

```

Frame
or
ParseError

```

5. `std::expected<void, E>`

并非所有成功操作都有返回值。

例如：

```

std::expected<void, SaveError>
save(const Document& document);

```

表达：

```

success
or
SaveError

```

比：

```

bool save(...);

```

提供更丰富的失败语义。

二、错误类型与传播

6. Error Type 设计

`expected` 的质量主要取决于：

E 的设计

而不是 `expected` 本身。

最简单：

```
enum class ParseError {
    truncated,
    invalid_header,
    unsupported_version,
};
```

适合：

- 错误类别有限
- 不需要额外 `context`
- 错误对象需要非常轻量

6.1 带结构化 Context

如果 `recovery` 或 `diagnostics` 需要更多信息：

```
struct ParseError {
    enum class Code {
        truncated,
        invalid_field,
        unsupported_version,
    };

    Code code;
    std::size_t offset;
};
```

这比：

```
std::string error;
```

更适合机器决策。

7. Error Identity 与 Diagnostics 分离

推荐：

```
structured code
+
optional diagnostic context
```

例如：

```
struct FileError {
    FileErrorCode code;
    std::filesystem::path path;
};
```

其中：code

用于：

```
branching
recovery
metrics
```

而: path / message

用于:

```
diagnostics
logging
```

不要让:

```
if (error.message == "file not found")
```

成为程序控制流。

8. Error Type 应位于正确抽象层

底层可能产生: ECONNRESET

业务层真正关心: RepositoryError::unavailable

因此:

```
OS error
  ↓
transport error
  ↓
repository error
```

可以发生 translation。

但只有:

抽象语义真正变化时才翻译。

不要每一层机械包装一次错误。

9. 返回 std::unexpected

典型模式:

```
std::expected<Frame, ParseError> parse_frame(...) {
    if (input.empty()) {
        return std::unexpected(ParseError::truncated);
    }

    return Frame{...};
}
```

成功值:

```
return Frame{...};
```

错误：

```
return std::unexpected(error);
```

让两个通道在类型上明确区分。

10. 手动传播

C++23 没有 Rust ？一样的语言级传播操作符。

典型：

```
auto header = parse_header(input);

if (!header) {
    return std::unexpected(header.error());
}

return decode(*header);
```

重要的是：

不要在每一层为了传播而重新创造无意义错误。

如果错误类型相同，直接传播即可。

三、组合条件与状态责任

11. Monadic Composition

`and_then` 的回调返回 `expected`，而且其 `error_type` 必须与当前对象相同；`or_else` 则须保留 `value_type`。还要根据对象的 `cv/ref` 类别检查回调参数以及成功值、错误值的构造要求。它们不是任意类型或错误域的自动连接器。N4950: `expected.object.monadic`

操作	用途	不应忽略的条件
<code>transform</code>	转换成功值	错误传播及结果类型必须可构造
<code>and_then</code>	下一步也返回 <code>expected</code>	保持 <code>error_type</code>
<code>transform_error</code>	显式转换错误域	转换后的错误类型及成功值传播必须有效
<code>or_else</code>	失败分支返回另一个 <code>expected</code>	保持 <code>value_type</code> ；是否恢复成功仍由策略决定

完整正例 T18 使用纯内存替身展示 `read` → `parse` → `validate`，**不测试文件 I/O**。只有 `read` 的低层错误在边界转换，后续两步统一使用 `ConfigError`：

完整实验 · T18 · sample.cpp

```

#include <expected>
#include <string_view>

enum class FileError { missing };
enum class ConfigError { input, parse, invalid };
struct Config { int port; };

std::expected<std::string_view, FileError> read_file(std::string_view path) {
    if (path == "missing") return std::unexpected(FileError::missing);
    if (path == "bad") return "not-a-number";
    if (path == "zero") return "0";
    return "8080";
}

std::expected<Config, ConfigError> parse_config(std::string_view data) {
    if (data == "8080") return Config{8080};
    if (data == "0") return Config{0};
    return std::unexpected(ConfigError::parse);
}

std::expected<Config, ConfigError> validate_config(Config value) {
    if (value.port == 0) return std::unexpected(ConfigError::invalid);
    return value;
}

std::expected<Config, ConfigError> load_config(std::string_view path) {
    return read_file(path)
        .transform_error([](FileError) { return ConfigError::input; })
        .and_then(parse_config)
        .and_then(validate_config);
}

int main() {
    auto good = load_config("good");
    auto missing = load_config("missing");
    auto bad = load_config("bad");
    auto zero = load_config("zero");
    return good && good->port == 8080
        && !missing && missing.error() == ConfigError::input
        && !bad && bad.error() == ConfigError::parse
        && !zero && zero.error() == ConfigError::invalid ? 0 : 1;
}

```

若用 `or_else` 选择备用配置，当前层必须有资格决定 `fallback`，并检查备用操作本身的失败。不同 E 直接串联的编译失败反例与最小转换正例见 [T08 / T09](#)。

12. `.value()` 与 `operator*`

本节按 C++23 最终草案 N4950 的 `expected<T, E>`（非 `void`）重载说明：

访问	条件与失败行为
<code>value() &/value() const &</code>	Mandates 要求 E 可复制构造；无值时构造并抛出 <code>bad_expected_access<E></code>
<code>value() &&/value() const &&</code> <code>operator*/operator-></code>	同样要求 E 可复制，另须能从该重载的 <code>std::move(error())</code> 构造 E 使用前须已确立 <code>has_value() == true</code> ；不是带运行时错误报告的检查访问
<code>error()</code>	使用前须已确立 <code>has_value() == false</code>

这些类型要求不因当前对象恰好含值而消失。构造异常对象时复制 / 移动 E 本身也可能抛异常，因此不能把可观察异常集合概括为只有 `bad_expected_access<E>`。N4950: `expected.object.obs`、LWG 3843

对于 §16 的 `move-only` 错误对象，先检查状态再访问相应分支，不靠 `std::move(result).value()` 绕过规范要求。完整正反例为 T06 ~ T10 和 T16。

附件报告观察到 `libstdc++ 14` 接受 T16；这是相对于所选 N4950 基线的实现差异，不是可移植许可。本机结果另记在修订与验证记录，不覆盖历史观测。

13. `expected` 不是引用容器

C++23 不允许：

```
std::expected<T&, E>
```

实例化 `expected` 的 `value type` 不能是引用类型。

如果需要表达：reference or error

可以考虑：

```
std::expected<std::reference_wrapper<T>, E>
```

或者根据 `ownership/lifetime` 语义使用：

```
T*
```

但必须明确：

```
nullability
lifetime
ownership
```

14. `expected` 不自动意味着 `noexcept`

例如：

```
std::expected<std::string, Error> load();
```

内部仍然可能因为：

```
allocation
copy
move
error construction
```

抛异常。

甚至 `expected` 自己的某些操作也依赖 `T` 和 `E` 的构造/移动属性。

因此: `value-based failure`

和: `exception-free implementation`

不是同义词。

15. `expected` 不自动提供 Strong Guarantee

```
std::expected<void, Error> update(State& state);
```

只说明: `failure transport = expected`

并没有说明: `state after failure`

可能是:

```
unchanged
valid but modified
partial commit
```

必须单独写入 API contract。

16. Ownership 必须随失败语义明确

按值接收 `std::unique_ptr<Job>` 意味着调用时所有权进入参数。失败后是销毁、保留在队列，还是返还调用者，必须属于 API 契约。

若失败时要同时返还原因与任务，可把任务放进错误对象。完整正例 T19 只模拟拒绝入队，不实现真实队列：

完整实验 · T19 · sample.cpp

```
#include <expected>
#include <memory>
#include <utility>

struct Job { int id; };
enum class SendError { queue_full };
struct SendFailure {
    SendError error;
    std::unique_ptr<Job> job;
};

std::expected<void, SendFailure> reject(std::unique_ptr<Job> job) {
    return std::unexpected(SendFailure{SendError::queue_full, std::move(job)});
}
```

完整实验 · T19 · sample.cpp · continued

```

}
int main() {
    auto job = std::make_unique<Job>(Job{7});
    auto* identity = job.get();
    auto result = reject(std::move(job));
    if (job || result) return 1;
    auto failure = std::move(result.error()); // 已知当前是错误分支。
    job = std::move(failure.job);
    return failure.error == SendError::queue_full
        && job.get() == identity && job->id == 7 ? 0 : 2;
}

```

错误对象因此是 `move-only`。继续组合时也必须检查所选重载能否移动 / 复制错误；`value()` 的特殊要求见 §12。这里的所有权保证来自具体实现，不是 `expected` 自动提供的。

四、成本、反例与回查

17. Value-Based Failure 的成本模型

值通道的对象成本可以用“分支判别信息 + T/E 的内部存储”理解，但不是标准布局公式：大小还受对齐、填充、空类型及实现策略影响，不能直接用 `tag + max(sizeof(T), sizeof(E))` 推算 ABI。

`expected` 在自身对象中保存值或错误；T/E 的构造、复制和诊断字段仍可能分配或抛出。热路径可考虑小 `enum/code` 与 `offset/id`，在边界附加详细上下文。这里是成本分析方法，没有 `benchmark` 结果，也不以值通道必然快于异常作为结论。

18. Value-Based Failure 的适用区域

优先考虑：

```

high-frequency failure
parsing
validation
lookup
network protocol outcomes
storage operations
explicit business failures

```

尤其适合：

调用者自然需要立即根据失败类别分支。

19. Anti-Patterns

所有东西都返回 `expected`

错误：

```
programmer invariant violation
→ expected<..., InternalBug>
```

这可能隐藏 bug。

`optional` 吞掉错误原因

```
network timeout
→ nullopt
```

丢失重要语义。

`string` 作为错误身份

```
std::expected<T, std::string>
```

可以用于小型应用，但不应默认成为大型系统错误协议。

Error Type 绑定底层实现

业务 API：

```
std::expected<User, int /* errno */>
```

泄漏系统实现。

`.value()` 到处使用

如果逻辑已经围绕 `expected` 设计，应正常分支或组合，而不是：

```
auto x = result.value();
```

把 value-based failure 再偷偷转换成 exception。

20. FM-2 Review Checklist

先用公共 C1–C8 合同检查完整操作，再用以下问题回查本章机制。

- [] `absence` 与 `failure` 是否区分？
- [] `bool` 是否足够表达调用者所需信息？
- [] `sentinel` 是否有更清晰的类型替代？
- [] `expected` 的 `E` 是否结构化？
- [] `error identity` 与 `diagnostic text` 是否分离？
- [] `error type` 是否位于正确抽象层？
- [] `error translation` 是否真正改变语义？
- [] `expected` 是否被误认为 `noexcept`？
- [] `failure state guarantee` 是否单独定义？
- [] `failure` 后 `ownership` 是否明确？
- [] 高频路径 `error object` 是否足够轻量？
- [] `monadic chain` 的 `cv/ref`、值 / 错误类型与构造要求是否满足？
- [] `move-only` 错误对象是否避免了不满足 `Mandates` 的 `value()` 调用？
- [] `monadic chain` 是否保持了清晰的 `recovery boundary`？

21. FM-2 核心不变量

optional 表达 absence, expected 表达 success-or-failure。

Value-based failure 只定义 transport, 不自动定义状态保证。

Error 应该首先是结构化机器语义, 字符串主要用于 diagnostics。

Failure type 必须与 abstraction boundary 对齐。

Error transport 与 ownership 必须一起设计。

FM-3 · 异常传播语义

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-2](#) · [下一章：FM-4](#) · [术语与审查约定](#)

阅读入口

主讲异常对象、捕获、重抛与实际展开。资源和业务状态是否安全，分别由 FM-4/FM-6 的合同补足。

主阅读线。 §1-9 → §13-15 → §20-22；对象构造失败细节回查 FM-6。

失败契约。 传播路径要标出可匹配 handler、非抛出边界和清理动作；错误翻译与日志本身可能产生新失败。没有 handler 的路径不承诺完整析构。

证据边界。 这里主要是语言条款与路径推理；T13 的构造计数及 T11 的受控终止只有各自声明的覆盖。无 fm-test 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、抛出对象与匹配
- 二、展开、重抛与中间层
- 三、捕获与边界
- 四、选型、成本与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、抛出对象与匹配

0. 文档定位

本章主讲 C++ 异常的非局部传播，而不重新定义失败原因或状态保证。沿一条能到达匹配 handler、且展开过程未再失败的路径，可用下图理解：

throw → 初始化异常对象 → 寻找 handler → 展开相应作用域 → catch

中间层可以不改写返回类型，由实际展开执行已构造对象的析构。但没有 handler、触及非抛出边界或展开中再失败时，不能套用这条正常清理路径；具体限制在 §5。清理与业务回滚也不是同一件事。

1. Exception 是非局部控制流

例如：

```
void c() {  
    throw ParseError{};  
}  
  
void b() {  
    c();  
}  
  
void a() {  
    b();  
}
```

如果 a() 外层存在 handler：

```
c  
↓  
b skipped  
↓  
a skipped  
↓  
handler
```

中间层不需要：

```
检查返回码  
再返回返回码
```

这是 exception 与 value-based failure 最明显的控制流差异。

2. Exception Object

执行：

```
throw Error{...};
```

C++ 会建立一个 exception object。

它独立于：throw expression 中局部对象的普通 lifetime
并在匹配 handler 处理期间保持存在。

因此通常：

```
catch (const Error& error)
```

是正确方式。

3. Catch by Reference

推荐：

```
catch (const std::exception& error) {  
    ...  
}
```

而不是：

```
catch (std::exception error) {  
    ...  
}
```

后者可能产生：

```
copy  
+  
object slicing
```

丢失动态异常类型信息。

4. Handler 顺序

更具体类型必须放在前面：

```
try {  
    ...  
} catch (const ParseError& error) {  
    ...  
} catch (const std::exception& error) {  
    ...  
}
```

如果先：

```
catch (const std::exception&)
```

后面的派生类 handler 可能永远无法匹配。

二、展开、重抛与中间层

5. Stack Unwinding

Exception 被抛出后：当前正常控制流终止

下面讨论控制流转向匹配 handler 且发生栈展开的路径；不包括所有终止情形。

每个已经完成构造的 automatic object：按正常作用域逆序析构

例如：

```
void process() {
    File file;
    Lock lock;
    Buffer buffer;

    decode(); // throws
}
```

展开：

```
~Buffer
~Lock
~File
```

这就是 `exception` 与 `RAII` 能够组合的根本原因。

无匹配 `handler` 时，是否在终止前展开栈由实现定义；搜索 `handler` 触及非抛出异常规格函数的最外层时，完整、部分或不展开也由实现定义。不能依靠这些终止路径执行所有自动对象析构，更不能将 `terminate` 当作安全关机协议。N4950: [except.terminate/2](#)

6. 未完成构造的对象

例如：

```
class Session {
    File file_;
    Socket socket_;
    Buffer buffer_;
};
```

在 `Session` 的非委托构造过程中，假定 `buffer_` 的非委托初始化未完成就抛异常：

```
file_   constructed
socket_ constructed
buffer_ throws
```

则：

```
socket_ destroyed
file_   destroyed
```

但：`Session` 本身没有完成构造

因此不会执行：

```
Session::~~Session()
```

上述例子是非委托构造失败。若目标构造已成功，随后委托构造函数体抛异常，则会调用完整对象的析构函数；不要把两种情况混同。N4950: [except.ctor/4](#)

详见 [FM-6 §3](#) 与 [T13](#)。

7. Rethrow

当前 handler 中：

```
catch (const Error&) {
    record_context();
    throw;
}
```

表示：重新抛出当前 exception object

保留原异常动态类型。该片段还依赖 record_context() 不以新异常中断处理；若它抛出，原异常并不会按计划执行到 throw；。报告失败政策应与 FM-8 §2 一起审查。

不要无意义写：

```
catch (const Error& error) {
    throw error;
}
```

这会构造新的 throw operand，并可能：

```
copy
slice derived type
```

所以重新传播当前异常：

```
throw;
```

才是标准模式。

8. Exception Translation

有时需要：

```
low-level exception
→ domain exception
```

例如：

```
try {
    storage_.read();
} catch (const StorageError& error) {
    throw ConfigLoadError{...};
}
```

只应在：abstraction semantics changes

时进行。

否则保持 exception neutrality：

不理解这个错误的层不要捕获它。

9. Exception Neutrality

一个高质量中间组件可以：

不 `catch`
但仍然 `exception-safe`

例如：

```
void process() {  
    Buffer buffer;  
    transform(buffer);  
}
```

即使 `transform()` 抛异常：

RAII handles cleanup
exception propagates upward

该函数完全可以做到：

`exception-neutral`
+
`resource-safe`

这比：

```
catch (...)
```

后不知道如何处理更好。

三、捕获与边界

10. `catch (...)`

`catch (...)` 的典型合理位置是：

```
thread boundary  
plugin boundary  
event-loop boundary  
process top-level boundary  
foreign ABI boundary
```

而不是业务代码到处：

```
try {  
    ...  
} catch (...) {  
}
```

特别禁止：

```
catch all
+
silently continue
```

11. `std::exception_ptr`

当 `exception` 需要跨越：

```
thread
callback
deferred execution
```

保存时：

```
std::exception_ptr
```

是标准机制。

捕获：

```
auto error = std::current_exception();
```

之后：

```
std::rethrow_exception(error);
```

可以重新进入 `exception propagation`。

这比尝试：复制未知动态异常对象

可靠得多。

12. Exception 与 Constructor

构造函数无法通过普通返回类型表达：

```
std::expected<Object, Error>
```

因此：

```
Object::Object(...)
```

无法建立 `invariant` 时，`throw` 是自然语言机制之一。

另一种设计：

```
class Object {
public:
    static std::expected<Object, Error> create(...);

private:
    explicit Object(Resource resource) noexcept;
};
```

两种都合理。

选择依据：

```
project failure policy
failure frequency
construction semantics
API composition
exception policy
```

13. Exception 与 Destructor

Destructor 属于 cleanup infrastructure。

基本工程原则：

Destructor 不应允许异常逃出。

特别是在 stack unwinding 中：

```
已有一个 active exception
+
destructor 再抛一个异常逃出
```

会进入：

```
std::terminate()
```

因此可能失败的资源关闭操作应考虑提供：

```
close()
flush()
commit()
```

显式 API。

析构只做：best-effort non-throwing cleanup

14. Exception Specification 与 noexcept

exception 可能从函数逃出与：

```
noexcept
```

直接相关。

```
void f() noexcept;
```

不是说：内部绝不会 throw

而是承诺：exception cannot escape f()

若违反：std::terminate()

详细模型在 FM-5。

15. Exception Boundary

Exception 不应无设计地穿过：

```
thread entry
C ABI
plugin ABI
shared-library ABI with incompatible runtime
process boundary
RPC boundary
```

在这些位置需要：

```
catch
→ translate
→ report
```

四、选型、成本与回查

16. Exception 适合什么失败

通常更适合：

```
rare failure
deep propagation
constructor failure
operations where intermediate layers cannot recover
failure path where manual plumbing adds substantial noise
```

17. Exception 不适合什么

不推荐用于：

```
lookup miss
queue empty
normal EOF
frequent parse rejection
try_lock failed
hot-path expected branch
```

因为这些本质是：ordinary domain control flow

18. Exception 成本模型

不能简单说：exception 有成本

或者：exception zero-cost

准确模型取决于：

```

compiler
platform ABI
optimization level
exception frequency
code layout
binary size
unwind tables

```

很多主流 native ABI 会显著偏向：

```

success path relatively cheap
throw path expensive

```

因此工程策略应是：

按 **failure semantics** 选择机制，对真正性能敏感路径进行测量。

而不是根据口号决定。

19. Exception Hierarchy

不要设计几十层 inheritance hierarchy。

典型项目：

```

class ApplicationError : public std::runtime_error {
    ...
};

```

再细分少量真正有语义差异的类型即可。

如果恢复策略实际上依赖：code

那么：single exception type + structured error code

有时比复杂继承树更合理。

20. Anti-Patterns

Catch and ignore

```

try {
    update();
} catch (...) {
}

```

Catch too low

底层没有 recovery context 却捕获所有异常。

Exception 作为普通循环分支

```
for (...) {
    try {
        lookup();
    } catch (NotFound&) {
    }
}
```

Throw raw primitives

```
throw 42;
throw "error";
```

破坏统一 error vocabulary。

Destructor 传播异常

容易破坏 unwinding。

每层重新包装 exception

产生：

```
ExceptionA
→ ExceptionB
→ ExceptionC
```

却没有新的抽象意义。

21. FM-3 Review Checklist

先用公共 C1-C8 合同检查完整操作，再用以下问题回查本章机制。

- [] failure 是否真正适合 exceptional propagation?
- [] exception 是否被 catch by reference?
- [] catch 顺序是否由具体到通用?
- [] 中间层是否可以保持 exception-neutral?
- [] rethrow 是否使用 throw;?
- [] stack unwinding 是否由 RAII 正确清理?
- [] destructor 是否可能传播异常?
- [] translation 是否发生在真正 abstraction boundary?
- [] exception 是否可能逃出 thread / ABI boundary?
- [] catch(...) 是否位于明确 failure boundary?
- [] failure frequency 是否适合 exception?

22. FM-3 核心不变量

Exception 是传播机制，不是 failure taxonomy。

Exception safety 不要求 catch exception。

RAII 是 exception propagation 能够安全组合的基础。

不知道如何恢复的中间层通常应该保持 exception-neutral。

Exception 必须在明确 boundary 内被 containment。

FM-4 · 资源清理与状态保证

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

返回 FM 导航 · 上一章：FM-3 · 下一章：FM-5 · 术语与审查约定

阅读入口

唯一主讲 strong/basic/no-throw/no-fail 的区别、prepare-commit-cleanup 和可观察状态。其他章节引用本章，不另设保证等级。

主阅读线。§1-7 → §10-17 → §18-19；用 §7 的状态表检查每个 mutation。

失败契约。先限定受保护状态，再枚举准备、提交、返回、清理各阶段。RAII 管理责任，不能代替业务回滚、远端确认或持久化协议。

证据边界。状态表与 swap 片段是条件化论证，不是已执行的全部失败注入；T17 只验证所列 Buffer 值语义，未注入分配失败。无 fm-test 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、所有权与保证语言
- 二、事务式状态转换
- 三、清理与外部副作用
- 四、审查与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、所有权与保证语言

0. 文档定位

FM-4 解决：

一个操作失败以后，资源和状态如何仍然保持正确？

核心关系：

```
RAII
+
ownership
+
state guarantees
+
transactional update
```

共同构成 C++ failure safety。

1. RAII 的真正含义

资源获取即初始化（Resource Acquisition Is Initialization，RAII）将清理 / 释放责任绑定到对象生命周期。Owner 可以管理内存、文件描述符、socket、锁、映射、GPU 资源或订阅，而不只是一种智能指针写法。

它统一正常返回、提前返回和实际展开中的资源清理；清理动作的成功条件、对象是否已完成构造，以及终止路径仍须分别分析。RAII 不自动恢复业务状态，也不保证关闭、flush 或持久化提交成功。

2. Owner Object

资源进入已构造的 owner 后，生命周期负责在应执行析构的退出路径调用清理：

获取资源 → 已构造 owner → 使用 → 正常退出 / 实际展开 → 析构清理

获取后尚未进入 owner 的窗口需要单独保护。进程终止、std::exit 等路径不能泛化成“所有自动对象都析构”；尤其本系列的终止与展开限制见 FM-3 §5 及 N4950 终止设施。

3. RAII 与 Control Flow 解耦

错误：

```
auto* resource = acquire();

step_a();
step_b(); // may throw

release(resource);
```

如果：step_b throws

release 被绕过。

正确：

```
Resource resource = acquire();

step_a();
step_b();
```

cleanup 成为：lifetime semantics

而不是：control-flow bookkeeping

这里以正常作用域退出或实际进行的栈展开为前提，不是所有终止路径的清理承诺；详见 [FM-3 §5](#)。

4. Exception Safety Guarantees

以下是工程保证的术语，不应混成一个包含“终止”的线性等级。声明保证时应明确所覆盖的可观察状态、失败通道与前置条件。

No-throw Guarantee

本系列用 `no-throw` 描述**不向调用者传播异常**的保证；它不等于 `no-fail`。`noexcept` 也不证明操作成功、正常返回或完成清理。

若一个 `commit` 要求 `no-fail`，必须额外证明：合法前置条件下可正常完成，没有未处理的错误返回、异常或部分提交，后续返回与清理不会推翻声明的事务结果。终止进程不算完成这样的 `commit`。

Strong Guarantee

所约定的失败发生时，可观察状态与操作前一致。典型实现先准备临时状态再提交；外部副作用不因内存中的 `swap` 自动回滚。

Basic Guarantee

失败后不变量和资源管理仍成立，但值可能已改变。具体可继续执行哪些操作仍须由契约说明。

No Useful Guarantee

对于关注的失败路径，缺少可依赖的状态保证，不能自行升级成 `basic`。某条标准写 `effects unspecified` 也不能简单改称 UB，见 [FM-5 §11](#)。

终止是另一维度的处置策略；其清理限制见 [FM-3 §5](#)。

二、事务式状态转换

5. Strong Guarantee 的核心模式

```
prepare (may fail, original observable state unchanged)
  ↓
commit (normal completion assured under its preconditions)
  ↓
return / cleanup (compatible with the promised guarantee)
```

契约片段：

```
void update(State& state, const Input& input) {
    State next = build_state(state, input);
    state.swap(next);
}
```

此结构只有在以下条件成立时才支持所声称的 `strong guarantee`：

- `build_state` 的失败不修改纳入保证的原状态，也没有无法撤销的外部副作用。
- `swap` 的前置条件满足；它真正完成提交，不能只是声明 `noexcept` 却在内部失败、终止或错误返回。

- `next` 的析构、返回路径及其他收尾与保证兼容；不能在提交后又向调用者报告一个声称“状态未变”的失败。

因此 `noexcept swap` 是有用线索，不是独立的事务证明。

6. Transactional Thinking

状态修改操作应思考：

```
Prepare
Validate
Commit
Cleanup
```

而不是：

```
边修改
边验证
边申请资源
边做不可逆副作用
```

后者极难提供 `strong guarantee`。

7. Commit Point

提交点（`commit point`）是合同把准备结果接受为新状态的边界。它不是异常处理机制，也不自动证明原子性、持久化或跨线程可见性。

以 \$5 的隔离准备 + `swap` 为例，下表是**条件化推理**，不是已执行的故障注入记录：

阶段	状态与责任	失败时可以声称什么
S0：进入	<code>state</code> 满足不变量，明确受保护的观察范围	前置条件不成立的调用不在此证明内
S1： <code>build_state</code>	<code>next</code> 独立构造； <code>state</code> 尚未被改动	只有构造及清理满足条件，才可丢弃临时状态并保持旧 <code>state</code>
S2： <code>swap</code> 提交	满足 <code>swap</code> 前提，且它确实完成、不半途报错	成功后新 <code>state</code> 生效； <code>noexcept</code> 一项不足以证明 <code>no-fail</code>
S3：返回 / 清理	旧状态由 <code>next</code> 管理，调用者接收完成结果	若仍能报告失败，就不能同时许诺该失败意味着旧状态不变

因此，提交前“旧状态保持”必须由隔离准备证明；提交后“新状态有效”必须由提交与收尾证明。函数若在提交后返回错误，应明示已提交或完成未知，而不是复用“明确未提交”的错误。

文件写入、消息发布和数据库操作也需找边界，但各自的持久化、部分副作用及确认协议不同。它们只是分析对象，不是用来证明内存 `swap` 合同的类比。

8. Partial Construction

RAII 的强大之处在于: subobjects constructed one by one

某一步失败后: already-constructed objects unwind automatically

委托构造体失败与普通成员构造失败的区别见 FM-6 §3。

因此应优先:

```
class Session {  
    File file_;  
    Socket socket_;  
    Buffer buffer_;  
};
```

而不是:

```
class Session {  
    FileHandle file_{invalid};  
    SocketHandle socket_{invalid};  
  
    bool init();  
};
```

后者创造大量: partially initialized states

9. Two-Phase Initialization

反模式:

```
Widget widget;  
if (!widget.init()) {  
    ...  
}
```

现在 Widget 存在:

```
before init  
after init  
init partially failed
```

多个合法/非法状态。

如果可行, 更推荐:

```
auto widget = Widget::create(...);
```

或者 throwing constructor。

目标:

一个普通对象一旦存在, 就满足其 invariant。

10. Rollback 不是免费操作

不要简单设计：

```
do A
do B
do C

C failed

undo B
undo A
```

因为：undo B 也可能失败

高质量 strong guarantee 更偏向：

```
build isolated new state
+
non-failing commit
```

而不是依赖复杂 rollback。

11. Copy-and-Swap

经典形式：

```
Buffer& operator=(Buffer other) {
    swap(other);
    return *this;
}
```

模型：

```
construct temporary
    ↓ may fail
old object unchanged

swap
    ↓ noexcept
commit

temporary destroys old state
```

这是非常清晰的 strong-guarantee 教学模型。

但不是所有类型都应该机械使用：

```
temporary allocation
capacity reuse lost
performance tradeoff
```

需要实际评估。

三、清理与外部副作用

12. Resource Acquisition 顺序

多个资源：

```
File file = open_file();
Socket socket = open_socket();
Buffer buffer = allocate_buffer();
```

每一步成功后立刻进入 owner。

那么：buffer acquisition fails

之前的：

```
socket
file
```

都会由 RAII 自动清理。

不要：

```
先获取三个 raw handle
再统一包装
```

这样会扩大 failure window。

13. Destruction 是 Cleanup Infrastructure

Destructor 应：restore resource ownership balance

而不是承担复杂可能失败的业务事务。

如果：

```
flush
commit
network close handshake
database commit
```

的失败需要调用者知道，应提供：

```
std::expected<void, CloseError> close();
```

析构函数只做：non-throwing fallback cleanup

14. State Guarantee 必须组合分析

高层操作的保证依赖：sub-operation guarantees

例如：

```

Algorithm guarantee
=
allocation guarantee
+
T construction guarantee
+
T move/copy guarantee
+
commit operation guarantee

```

因此 generic code 不能脱离 T 的 semantics 宣称 guarantee。

15. Failure 与 Partial Side Effects

内存临时状态可以在未发布时丢弃；邮件已发送、付款已提交或消息已被确认，却可能没有恢复原状的逆操作。因此必须先限定 strong guarantee 所覆盖的状态，再描述外部效果。

回滚（rollback）恢复约定旧状态；补偿（compensation）用新动作补救已发生的效果，不保证历史被抹去，补偿自身也可能失败。无法保证全有或全无时，应明确部分进度、已提交范围及可重试条件，而不是把资源清理写成业务回滚。

16. Ambiguous Completion

典型：

```

request sent
server committed
response lost

```

调用者看到：timeout

但实际：operation may already have succeeded

这不是普通 strong/basic guarantee 能完全表达的。

需要：

```

idempotency
operation ID
deduplication
query-after-failure
transaction protocol

```

17. RAII 不等于 Transaction

RAII 可以保证：resource cleanup

但不能自动保证：business state rollback

例如：File handle successfully closed

并不能回滚：已经写入磁盘的数据

两者必须分开。

四、审查与回查

18. FM-4 Review Checklist

先用公共 C1-C8 合同检查完整操作，再用以下问题回查本章机制。

- [] 当前组件负责释放的 `resource` 是否立即进入明确的 `owner`，借用是否保持 `non-owning`？
- [] `raw ownership window` 是否最小？
- [] 对象是否存在 `partially initialized state`？
- [] 异常传播、失败后状态和终局处置是否分别声明？
- [] `guarantee` 是否依赖 `T` 的操作？
- [] `commit point` 是否明确？
- [] `prepare` 阶段是否修改原状态？
- [] `commit` 是否真正 `no-fail` (含前置条件、错误返回与收尾)，而不只是 `noexcept`？
- [] `rollback` 本身是否可能失败？
- [] `destructor` 是否承担可失败业务逻辑？
- [] `external side effect` 是否存在 `ambiguous completion`？
- [] `failure` 后 `ownership / release responsibility` 是否仍然明确 (包括共享所有权)？

19. FM-4 核心不变量

当前组件负责释放的资源，应由明确的 `owner / resource handle` 将释放责任绑定到对象生命周期；借用资源必须明确保持 `non-owning`。

这是资源责任的工程建议，不要求当前组件拥有所有访问到的资源；参见 [C++ Core Guidelines R.1](#) 与 [R.3](#)。

`Strong guarantee` 的首选模型是 `prepare-then-commit`，而不是复杂 `rollback`。

`Basic guarantee` 至少要求 `invariant` 与 `ownership` 仍然正确。

`RAII` 在适用生命周期路径履行资源清理责任；清理动作及其成功条件仍需审查，不自动保证业务事务原子性。

`External side effect` 必须单独定义 `commit semantics`。

FM-5 · 异常边界与泛型保证

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-4](#) · [下一章：FM-6](#) · [术语与审查约定](#)

阅读入口

核对 `noexcept`、移动 / 复制、`traits`、`swap` 与具体容器操作能承诺什么。状态保证的定义仍以 FM-4 为准。

主阅读线。 §1-5 → §7-13 → §16-19；不要从函数名或 `trait` 名称跳过完整表达式。

失败契约。调用前的参数构造、函数体、返回与临时对象清理分别可能失败。`noexcept` 禁止异常越界，不证明正常完成；复制 / 移动后的源与目标状态另行说明。

证据边界。T14 查完整表达式，T17 保留 R01 完整值 / 独立存储判据；T03 为复制回退。容器 `throwing-move` 注入矩阵仍未覆盖。无 `fm-test` 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、传播边界与表达式
- 二、移动与值复制
- 三、容器策略与类型条件
- 四、提交与组合保证
- 五、审查与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、传播边界与表达式

0. 文档定位

FM-5 解决：

C++ 如何把“这个操作不会让异常逃出”的信息编码进类型接口，并让泛型算法据此改变策略？

核心对象：

```
noexcept
move
copy
swap
type traits
container relocation
```

1. noexcept 的精确含义

```
void f() noexcept;
```

表示：

exception 不允许逃出 f()。

不是：f 内部绝对没有 throw

例如：

```
void f() noexcept {
    try {
        may_throw();
    } catch (...) {
        recover();
    }
}
```

这是契约片段：may_throw 和 recover 未给出实现。内部抛出再捕获合法，但若 recover() 又抛出并越界，仍会终止；此例不证明恢复动作不会失败。

2. 违反 noexcept

如果异常试图逃出：

```
void f() noexcept {
    throw Error{};
}
```

则：std::terminate()

因此：noexcept

是一项非常强的 runtime contract；它不保证正常完成或完整清理。终止与栈展开的边界见 FM-3 §5。

3. noexcept(expr)

noexcept(expression) 是不求值的编译期查询，判断整个表达式是否 potentially-throwing，不只是查看最外层被调函数有没有 noexcept。实参求值和临时对象等也参与判断。N4950: `expr.unary.noexcept`

编译语义例 T14:

完整实验 · T14 · sample.cpp

```
int argument();
void operation(int = argument()) noexcept;
static_assert(noexcept(operation(0)));
static_assert(!noexcept(operation()));
```

两次调用的函数相同，默认实参的求值却改变了结果。此例只需语法检查，不需要为未调用的函数补定义或链接入口。

4. Conditional noexcept

泛型代码:

```
template <typename T>
void exchange(T& a, T& b)
    noexcept(noexcept(a.swap(b)))
{
    a.swap(b);
}
```

这让高层 operation 的 exception specification: 由底层 operation 推导

二、移动与值复制

5. std::move 不执行移动

std::move(x) 执行表达式转换，保留相应 cv 限定，不负责转移资源。对于初始化片段:

```
T y{std::move(x)};
```

完整重载决议才决定使用哪个构造函数: 可能移动、可能复制，也可能因为被选函数删除 / 不可访问等而不合法。即使选中移动构造，资源效果仍由类型实现决定。N4950: [forward](#)

完整正例 T03 对照了没有移动构造的复制回退，以及 std::move(const_object) 选择复制。未声明移动构造与显式声明 **deleted** 移动构造不同; 后者可能在重载决议中胜出后使初始化失败。

6. Move 为什么特别适合 noexcept

典型 ownership type:

```
Buffer(Buffer&& other) noexcept
    : data_{std::exchange(other.data_, nullptr)},
      size_{std::exchange(other.size_, 0)} {}
```

只需要:

```
transfer handle
clear source
```

往往不需要：

```
allocate
clone
I/O
```

因此 `move` 是非常适合 `no-throw guarantee` 的操作。

7. Copy 与 Failure

复制拥有型缓冲区通常需要分配独立资源并复制内容。只写 `new std::byte[n]` 而不复制数据，不能实现这里约定的值复制。

完整正例 T17 与 §6 的裸指针片段是两个独立实现。本例用 `unique_ptr` 管理数组；空缓冲区和移出后的对象均为 `size 0`、空指针。复制赋值通过按值参数和交换实现；参数复制可能在进入函数体前失败。

完整实验 · T17 · sample.cpp

```
#include <algorithm>
#include <cstddef>
#include <memory>
#include <span>
#include <utility>

class Buffer {
    std::unique_ptr<std::byte[]> data_;
    std::size_t size_ = 0;
public:
    Buffer() = default;
    explicit Buffer(std::span<const std::byte> input)
        : data_{input.empty() ? nullptr :
            ↳ std::make_unique<std::byte[]>(input.size())},
          size_{input.size()} {
        if (size_ != 0) std::copy_n(input.data(), size_, data_.get());
    }
    Buffer(const Buffer& other)
        : data_{other.size_ == 0 ? nullptr :
            ↳ std::make_unique<std::byte[]>(other.size_)},
          size_{other.size_} {
        if (size_ != 0) std::copy_n(other.data_.get(), size_, data_.get());
    }
    Buffer(Buffer&& other) noexcept
        : data_{std::move(other.data_)}, size_{std::exchange(other.size_, 0)} {}
    Buffer& operator=(Buffer other) noexcept {
        swap(other);
        return *this;
    }
    void swap(Buffer& other) noexcept {
        data_.swap(other.data_);
```

完整实验 · T17 · sample.cpp · continued

```

        std::swap(size_, other.size_);
    }
    std::span<std::byte> bytes() noexcept { return {data_.get(), size_}; }
};

int main() {
    const std::byte input[]{std::byte{1}, std::byte{2}, std::byte{3}};
    Buffer original{input};
    Buffer copied{original};
    if (copied.bytes().data() == original.bytes().data()
        || !std::ranges::equal(copied.bytes(), input)) return 1;
    copied.bytes()[0] = std::byte{9};
    if (original.bytes()[0] != std::byte{1}) return 2;
    Buffer empty;
    Buffer empty_copy{empty};
    if (!empty_copy.bytes().empty() || empty_copy.bytes().data() != nullptr) return
        3;
    Buffer assigned;
    assigned = original;
    if (!std::ranges::equal(assigned.bytes(), input)) return 4;
    if (assigned.bytes().data() == original.bytes().data()) return 6;
    assigned.bytes()[0] = std::byte{8};
    if (original.bytes()[0] != std::byte{1}) return 7;
    Buffer moved{std::move(copied)};
    return copied.bytes().empty() && copied.bytes().data() == nullptr
        && moved.bytes()[0] == std::byte{9} ? 0 : 5;
}

```

分配仍可能抛出 `std::bad_alloc`；本例的字节复制本身不抛异常，已取得的资源由成员管理。T17 对复制构造和复制赋值检查完整长度与内容、存储独立及修改副本不影响源对象，另检查空对象和移出状态；不声称已经注入内存分配失败。这里使用比较两个完整区间的 `std::ranges::equal`，避免三迭代器版本只比较第一区间对应前缀而漏掉空值或截断复制。N4950: [alg.equal](#)

三、容器策略与类型条件

8. Container Relocation

假设：

```
old vector
```

```
[A][B][C]
```

扩容：new storage

需要迁移元素。

如果 copy：

```
copy A
copy B
copy C throws
```

旧：[A][B][C]

通常仍未改变。

新 storage 可以销毁。

这很有利于：strong guarantee

9. Throwing Move 的困难

如果：

```
move A
move B
move C throws
```

旧 storage 可能已经：[A_moved][B_moved][C]

此时：rollback

并不简单。

因为把对象 move 回去：可能再次 throw

10. std::move_if_noexcept

std::move_if_noexcept(value) 同样只产生引用，不执行构造或资源转移：

表 1 · 字段记录视图（列名与原单元格逐项对应）

表 1 · 记录 1	
类型条件	!std::is_nothrow_move_constructible_v<T> && std::is_copy_constructible_v<T>
返回类型	const T&

表 1 · 记录 2 · 其他情况	
类型条件	其他情况
返回类型	T&&

后续初始化再通过重载决议选择操作；返回 T&& 也不证明存在或调用了移动构造。该工具有利于泛型算法选择复制回退，但不独自证明算法的异常保证。N4950: forward

11. Move-Only + Throwing Move

删除复制而允许移动构造抛异常的类型没有复制回退。但“较弱保证”不能代替具体操作契约：

操作与条件	N4950 中可依赖的范围
<code>vector::reserve</code>	通常异常时无效果；非 <code>Cpp17CopyInsertable</code> 类型的移动构造抛异常，是该无效果保证的明确例外
<code>vector</code> 末尾插入单个元素， <code>T</code> 为 <code>Cpp17CopyInsertable</code> 或可不抛异常地从 <code>T&&</code> 构造	异常时无效果
插入条款的其他情形中，非 <code>Cpp17CopyInsertable</code> <code>T</code> 的移动构造抛异常	<code>effects unspecified</code> ，不能自行改写为 <code>basic guarantee</code>

`Cpp17CopyInsertable` 是针对容器及其 `allocator` 的要求，不能只用“有没有复制构造”替代完整判定。上述例外不自动等于 UB，也不授权读取未被保证的旧值来推断恢复策略。N4950: [vector.capacity/4](#)、[vector.modifiers/2](#)

本项是条款核对；定向测试没有穷尽容器异常注入或所有实现的失败后状态。

12. Traits

常见：

```
std::is_nothrow_move_constructible_v<T>
std::is_nothrow_move_assignable_v<T>
std::is_nothrow_copy_constructible_v<T>
std::is_nothrow_copy_assignable_v<T>
```

这些 traits 检查的是相应构造 / 赋值表达式的性质。例如 `is_nothrow_move_constructible_v<T>` 为真，也可能因为 `T&&` 绑定到不抛异常的复制构造，不能据此证明 `T` 存在移动构造。T03 给出反例。

它们不是纯 metaprogramming trivia。

它们描述：generic algorithm 可以依赖哪些 failure properties

四、提交与组合保证

13. swap

swap 经常作为：

commit primitive

因此：

```
void swap(T& other) noexcept;
```

非常有价值。

如果高层 strong guarantee 依赖：

```
prepare
→ swap commit
```

那么必须审查选中的 swap 的前置条件、异常与其他失败通道，以及 prepare 和收尾。

可抛 `swap` 不能直接作为这种无失败提交原语；反过来，声明 `noexcept` 也不是 `no-fail` 的证明。完整状态论证只在 [FM-4 §5-7](#) 维护，不能在本章缩成“一个 `noexcept swap` 就足够”。

14. Defaulted Move

如果类型主要由标准 RAII members 组成：

```
class Record {
public:
    Record(Record&&) = default;

private:
    std::string name_;
    std::vector<int> values_;
};
```

编译器生成 `move` 的 `exception specification` 会依据其 `bases/members` 的对应操作确定。

不要为了“性能”无条件手写：

```
Record(Record&&) noexcept = default;
```

除非成员语义确实支持这一承诺。

15. 不要对 `noexcept` 撒谎

错误：

```
Object(Object&& other) noexcept {
    resource_ = allocate_resource();
}
```

如果 `allocation` 抛异常：`terminate`

不是：`move returned an error`

因此：

`noexcept` 是 `correctness contract`，不是 `performance annotation`。

16. Destructor 与 `noexcept`

没有显式异常规格的析构函数，其隐式规格依赖潜在构造子对象等规则，不能一概说“所有析构默认不抛”。这是语言性质；工程上仍应设计可靠的非传播清理边界。[N4950: except.spec](#)

Destructor 应被设计为：`non-throwing cleanup boundary`

不要通过：`destructor exception`

报告：

```
flush failed
commit failed
close handshake failed
```

这些应在显式操作里报告。

17. Generic Guarantee Composition

可以形成：

```
high-level non-propagation proof
=
all expression paths, arguments, temporaries and cleanup checked
+
any internal exceptions handled without new escaping exceptions
```

以及：

```
high-level strong guarantee
=
algorithm structure
+
element operation properties
+
non-failing commit
```

第二个式子还要求 `prepare` 不改变受保护状态，以及提交后的返回和清理不违背保证；具体条件见 [FM-4 §5](#)。这两个式子是工程证明提纲，不是仅凭 `noexcept` 或 `traits` 就能完成的证明。

五、审查与回查

18. FM-5 Review Checklist

先用公共 [C1–C8](#) 合同检查完整操作，再用以下问题回查本章机制。

- ☐ `noexcept` 是否是真实语义承诺？
- ☐ `move` 是否真的只转移 `ownership`？
- ☐ `move` 是否进行了 `allocation/I/O/registration`？
- ☐ `swap` 是否能够 `noexcept`？
- ☐ `container relocation` 是否依赖 `nothrow move`？
- ☐ `throwing move` 是否还有 `copy fallback`？
- ☐ `defaulted special members` 是否比手写实现更可靠？
- ☐ `generic noexcept` 是否条件化？
- ☐ 是否把 `std::move` 错解为“执行移动”？
- ☐ 是否为了优化错误地扩大 `noexcept`？

19. FM-5 核心不变量

`noexcept` 表示 `exception` 不会逃出函数。

`std::move` 只改变 `value category`。

Nothrow move 是类型提供给泛型算法的重要 capability。

Throwing move 会限制 generic code 能提供的 strong guarantee。

noexcept 必须源于真实语义，而不是性能愿望。

FM-6 · 生命周期与资源失败

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-5](#) · [下一章：FM-7](#) · [术语与审查约定](#)

阅读入口

主讲对象何时建立不变量、部分构造清理、分配与初始化失败，以及显式完成和析构后备的分工。

主阅读线。 §1-5 → §7-13 → §14-17；RAII 的一般保证与提交分析回查 FM-4。

失败契约。 构造成功不变量成立；失败时哪些子对象已完成、谁拥有资源必须清楚。finish/flush 的业务完成与析构释放分开，nothrow 分配不等于整个 new-expression 不抛。

证据边界。 T13 只检查普通与委托构造路径的计数；没有穷举 allocator/new-handler、资源耗尽与实际关闭失败。无 fm-test 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- [一、构造与对象有效性](#)
- [二、分配与资源压力](#)
- [三、显式完成和清理](#)
- [四、审查与回查](#)

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、构造与对象有效性

0. 文档定位

FM-6 聚焦对象生命周期边界上的失败：

```

construction
allocation
partial construction
destruction
resource acquisition
factory

```

核心原则：

一个普通对象一旦成功存在，就应该满足其 **invariant**。

1. Constructor 是 Invariant Boundary

理想构造：

```
Connection connection{options};
```

正常返回意味着：Connection invariant established

如果无法建立：constructor must not successfully return

2. Constructor Throw

例如：

```

File::File(const Path& path) {
    handle_ = open_native(path);

    if (!handle_.valid()) {
        throw FileOpenError{...};
    }
}

```

这是合法且自然的 C++ construction failure。

3. Partial Construction

在完整对象的非委托构造中，若成员 A、B 已完成，而成员 C 的非委托初始化尚未完成就抛异常，已完成的 B、A 按逆序析构；此时不会调用 C 或完整对象自身的析构函数。C 内已经完成构造的子对象仍按相应规则清理。已取得资源必须由完成构造的成员或局部 owner 管理，不能寄望于完整对象的析构函数。

委托构造是不同情况：目标构造已成功，随后委托构造函数体抛异常，会调用该对象的析构函数。

N4950: `except.ctor/3 ~ 4`

对照片段：

```

Object() : Object(0) {
    throw Failure{}; // 目标构造已成功：随后调用 ~Object()。
}

```

完整析构计数例见 T13。这些规则描述构造失败转向 handler 的路径；不能推广为所有进程终止都会完成栈展开，见 FM-3 §5。

4. Factory + expected

如果项目采用 value-based failure:

```
class Connection {
public:
    static std::expected<Connection, ConnectError>
    connect(const Options& options);

private:
    explicit Connection(Socket socket) noexcept;
};
```

流程:

```
factory performs fallible setup
    ↓
success
    ↓
construct fully valid object
```

这是非常优秀的 C++23 模式。

5. Throwing Constructor vs Factory

优先考虑 throwing constructor:

```
construction failure rare
project uses exceptions
object cannot meaningfully exist invalid
ergonomic direct construction valuable
```

优先考虑 factory:

```
failure is expected
caller branches on error
project uses expected
failure category is domain-visible
construction requires asynchronous/multi-step workflow
```

不要建立:

```
throwing constructor = old
factory = modern
```

这种错误二分。

6. Two-Phase Initialization 应谨慎

```
Connection c;
if (!c.init()) {
    ...
}
```

会产生：

```
default state
initialized state
partially initialized state
failed state
```

并迫使每个方法问：is_initialized?

这通常扩大 state space。

优先：

```
construct valid object
or
do not produce object
```

二、分配与资源压力

7. operator new

普通分配：

```
auto* p = new T;
```

无法分配时：通常通过 `std::bad_alloc` 报告

这是 exception-based allocation failure model。

8. Nothrow Allocation

下述讨论限定标准非抛出分配形式；类特定分配函数和对象初始化应分别审查，不能从 `new (std::nothrow)` 的字面形式推出整个表达式不抛。

可以：

```
auto* p = new (std::nothrow) T;
```

分配失败时：nullptr

但这并不自动意味着：T construction cannot throw

分配成功后，初始化失败仍可传播异常；`new-expression` 会按其规则寻找匹配的释放函数。资源回收与对象是否构造完成是两件事，不能把返回空指针和构造抛异常合成同一种失败路径。N4950: `expr.new`

同时也不要为了“禁用异常”机械使用 `nothrow new`。

现代工程更推荐：

```
RAII owner
+
明确 allocation policy
```

而不是 `raw new`。

9. `std::bad_alloc`

对于普通通用软件：system is out of memory

通常不是一个容易局部恢复的问题。

因为处理错误本身可能还需要：

```
allocate strings
log buffers
construct error objects
```

因此不要假设：

```
catch (const std::bad_alloc&) {
    continue_normally();
}
```

一定能够恢复。

10. Memory Failure Policy

真正需要在内存压力下继续工作的系统，应提前设计：

```
preallocation
bounded pools
arena allocation
memory_resource
reserved emergency capacity
admission control
backpressure
```

而不是等 `bad_alloc` 发生后才临时思考恢复。

11. Allocation 与 API Guarantee

如果 API 内部可能：

```
vector growth
string growth
shared_ptr control block allocation
dynamic polymorphic allocation
```

那么即使函数里没有：

```
throw
```

它也可能传播 allocation exception。

三、显式完成和清理

12. Destructor 的职责

Destructor：

```
release ownership  
restore resource balance
```

典型：

```
~File() noexcept {  
    close_native(handle_);  
}
```

但现实中：

```
close  
flush  
fsync  
commit
```

可能失败。

如果失败对业务正确性重要：

```
auto result = file.flush();  
auto result = transaction.commit();
```

必须通过显式 operation 观察。

不要依赖 destructor 报告。

13. Explicit Close Pattern

例如：

```
class Writer {  
public:  
    std::expected<void, FlushError> finish();  
  
    ~Writer() noexcept {  
        cleanup_without_throwing();  
    }  
};
```

语义：

```
finish()
    → observable completion/failure

destructor
    → resource cleanup only
```

这能区分: business completion
和: resource destruction

14. Resource Owner 优先使用 Rule of Zero

理想:

```
class Session {
    std::unique_ptr<Resource> resource_;
    std::vector<std::byte> buffer_;
    std::string name_;
};
```

尽量让成员自己管理:

```
copy
move
destruction
```

从而业务类型通常无需手写 Rule of Five。但成员各自安全不自动证明跨成员不变量, 默认移动后的源状态、复制是否被删除以及字段之间的关系仍须核对; 主讲见 [FM-5](#)。

这就是: Rule of Zero

在 Failure Model 中的巨大价值。

15. Rule of Five 适用于真正 Ownership Primitive

如果你正在实现:

```
FileHandle
Socket
MappedMemory
RawBuffer
```

这类基础 ownership abstraction,

才通常需要显式设计:

```
destructor
copy constructor
copy assignment
move constructor
move assignment
```

并明确每一个操作:

can fail?
noexcept?
source state?
strong/basic guarantee?

四、审查与回查

16. FM-6 Review Checklist

先用公共 C1-C8 合同检查完整操作，再用以下问题回查本章机制。

- ☐ 普通对象存在是否意味着 invariant 成立？
- ☐ constructor failure 后成员是否由 RAII 自动清理？
- ☐ 是否存在不必要 two-phase initialization？
- ☐ throwing constructor 与 factory 是否符合项目 Failure Profile？
- ☐ allocation failure 是否被遗漏？
- ☐ 是否把 bad_alloc 当成容易恢复的普通错误？
- ☐ 内存受限系统是否有预分配/池/backpressure？
- ☐ destructor 是否可能传播异常？
- ☐ 可失败 commit/flush 是否有显式 API？
- ☐ ownership primitive 的 Rule of Five 是否完整？
- ☐ 业务类型是否尽量使用 Rule of Zero？

17. FM-6 核心不变量

成功构造的对象必须满足 invariant。

Constructor failure 必须依靠 member-level RAII 自动清理。

可恢复 construction failure 可以使用 factory + expected。

Destructor 是 cleanup boundary，不应承担可失败业务 commit。

资源受限问题应通过资源策略设计解决，而不是依赖 OOM 后临时恢复。

FM-7 · 系统错误与领域转换

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-6](#) · [下一章：FM-8](#) · [术语与审查约定](#)

阅读入口

主讲 `errno`、`error_code/error_condition`、`system_error` 与领域错误的身份转换；不由原生错误名称直接决定业务恢复。

主阅读线。 §1-7 → §10-14 → §15-16；值通道组合回查 FM-2，重试协议回查 FM-8。

失败契约。 先确认原接口失败再保存错误，保留 `category` 与必要 `cause`；错误翻译可能分配，`error_code` 参数也不自动封闭全部异常通道。

证据边界。 T15 只查询具体库签名；T20 检查 `open` 的返回值与选项。它们不穷举平台错误映射、路径分配或恢复策略。无 `fm-test` 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、原生错误身份
- 二、通道、领域与上下文
- 三、审查与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、原生错误身份

0. 文档定位

FM-7 解决系统编程中的底层失败表示：

```
errno
OS error
std::error_code
std::error_condition
std::system_error
domain translation
```

目标不是把整个程序都改成 `error_code`，而是：

正确把 OS failure 引入现代 C++ Failure Model。

1. `errno`

POSIX API 常见：

```
int fd = ::open(...);

if (fd == -1) {
    int error = errno;
}
```

关键规则：

只有在 API 明确定义失败并设置 `errno` 后，才读取它。

并且应尽快保存：

```
const int error = errno;
```

因为后续 library/system calls 可能改变它。

2. 不要把 `errno` 当全局 Error Object

错误：

```
call A
call B
...
later read errno
```

无法保证它仍对应 A。

正确模型：

```
detect failure
↓
capture errno immediately
↓
translate
```

3. `std::error_code`

核心结构可以理解为：

```
integer value  
+  
error category identity
```

因此：value = 2

本身没有足够语义。

需要：category

一起解释。

4. `error_code` 的价值

它非常适合：

```
OS failure  
library error domains  
non-throwing system APIs  
cross-layer low-level error transport
```

并具有：small value-like object

的工程性质。

5. `std::errc`

标准提供一组可移植的错误条件：

```
std::errc::permission_denied  
std::errc::timed_out  
std::errc::no_such_file_or_directory
```

它们让代码可以表达：portable condition

而不是绑定某个平台整数。

6. `error_code` vs `error_condition`

`error_code` 保存具体值及 category 身份；`error_condition` 表达一般化条件，是否等价由 category 的映射 / 比较语义决定。一个平台 socket 错误可以对应 `connection_refused`，但不能只比整数，也不能把任意 category 的同值错误视为同一种情况。

这是错误表示层的关系，不决定是否恢复或重试。错误对象有效，不代表被操作的文件、事务或远端状态也仍满足成功不变量。

二、通道、领域与上下文

7. `std::system_error`

当项目选择 exception propagation:

```
throw std::system_error{ec, "open socket"};
```

可以把: `std::error_code`

带入 exception channel。

于是:

```
representation
    error_code

transport
    exception
```

再次说明:

Error identity 和 transport mechanism 是两个不同维度。

8. Value-Based 系统 API

另一种:

```
std::expected<File, std::error_code>
open_file(const Path& path);
```

非常适合作为: low-level infrastructure API

但更高层可能应翻译:

```
std::expected<Config, ConfigError>
load_config(...);
```

而不是一路暴露 native error。

9. Error Domain

每个 library/subsystem 可以拥有自己的: error category/domain

例如:

```
filesystem
network
database client
protocol decoder
```

但不要因为技术上能建立 custom error_category 就全部使用。

简单领域错误:

```
enum class ParseError
```

可能更加清晰。

10. 底层错误不能直接决定业务策略

例如：ETIMEDOUT

不自动等于：retry

上层还要知道：

```
idempotent?  
deadline?  
retry budget?  
shutdown?  
operation already committed?
```

所以：system error

只是事实，

不是：recovery policy

11. Error Translation 示例

```
ECONNRESET  
  ↓  
HttpClientError::connection_lost  
  ↓  
RepositoryError::unavailable
```

到业务层：Repository unavailable

通常比：ECONNRESET

具有更稳定的抽象意义。

12. Preserve Cause

Translation 不应完全摧毁 diagnostics。

可以：

```
struct RepositoryError {  
    RepositoryErrorCode code;  
    std::error_code cause;  
};
```

或者在 logging/tracing context 中保留：native cause

这样兼顾：

```
business semantics
+
low-level diagnosis
```

13. Error Context

错误对象可以包含：

```
operation
resource identity
offset
endpoint
path
partition
```

但需注意：

```
PII
secret
credential
large allocation
```

不要无控制将敏感或巨大 context 塞进 error。

14. API 双版本模式

带 `error_code&` 的重载可按具体契约报告指定的操作错误，**不自动保证整个调用不抛异常**。文件系统接口的底层错误通道与其他异常条件应分别查看。[N4950: fs.err.report](#)

例如，[N4950](#) 的 `current_path(error_code&)` 查询重载返回 `path`，未声明 `noexcept`；设置重载 `current_path(const path&, error_code&)` 则声明了 `noexcept`。还须区别函数本身与构造实参、返回后使用结果的完整表达式。[N4950: fs.op.current.path](#)

[T15](#) 只用 `noexcept` 查询签名，不实际改变工作目录。查询版的负向结果是当前库观测；不将其强制为所有实现的要求。

项目是否提供双版本，应权衡接口面积、测试与文档成本，不必机械模仿标准库。

三、审查与回查

15. FM-7 Review Checklist

先用公共 [C1-C8](#) 合同检查完整操作，再用以下问题回查本章机制。

- [] `errno` 是否在失败后立即保存？
- [] 是否只在文档规定设置 `errno` 的失败后读取？
- [] `native error` 是否泄漏到不该知道它的业务层？
- [] `error_code` 是否比简单 `enum` 真有价值？
- [] `error translation` 是否改变了 `abstraction semantics`？
- [] `low-level cause` 是否保留用于 `diagnostics`？
- [] `timeout` 是否被错误地自动等价成 `retry`？
- [] `system_error exception` 是否符合当前 `exception policy`？
- [] `error context` 是否包含敏感信息？

16. FM-7 核心不变量

OS error 首先是事实，不是 `recovery policy`。

`error_code` 表示错误身份，`exception/expected` 决定传播方式。

Native errors 应在适当 `abstraction boundary` 翻译。

Error translation 应提升语义，同时尽量保留诊断 `cause`。

FM-8 · 执行边界与恢复协议

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-7](#) · [下一章：FM-9](#) · [术语与审查约定](#)

阅读入口

把局部失败放回线程、协程、回调、ABI、进程与远端操作中，明确观察点、报告后备、完成认知和恢复权限。

主阅读线。 §1-8 → §9-11 → §17-22；按实际集成边界回查 §12-16。

失败契约。 异常被捕获或存入 `future` 不代表业务已恢复。超时、取消请求、实际停止与提交是不同事实；确认丢失时不能假定未提交后盲重试。

证据边界。 T12 是 `join` 后观察的最小例，T11 是报告再失败的受控终止；未做 TSan、完整协程、ABI 兼容或分布式故障实验。无 `fm-test` 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、线程与异步观察
- 二、取消、超时与关闭
- 三、集成和远端边界
- 四、恢复协议与影响范围
- 五、审查与回查

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、线程与异步观察

0. 文档定位

FM-8 研究 `failure` 跨越执行边界时会发生什么：

```
thread
future
coroutine
callback
event loop
C ABI
plugin ABI
process
RPC
distributed side effect
```

这些位置必须建立：failure containment

而不是让失败无控制扩散。

1. Boundary 的本质

Boundary 表示：

当前 failure representation 不能或不应该原样继续传播的位置。

典型：

```
exception
↓
thread boundary
↓
must capture/translate
```

或者：

```
internal enum
↓
RPC
↓
serialized protocol error
```

2. std::thread

异常逃出线程初始函数会终止进程；线程入口的 catch 内若再次抛出，同样可能越界。因此隔离必须包括报告路径。N4950: `except.terminate`

完整正例 T12 只在线程内保存异常，在正常 join 后由协调者读取、重新抛出并处理：

完整实验 · T12 · sample.cpp

```

#include <exception>
#include <thread>
int main() {
    std::exception_ptr failure;
    std::thread worker([&failure] {
        try { throw 7; }
        catch (...) { failure = std::current_exception(); }
    });
    worker.join(); // Synchronizes before the read of failure.
    if (!failure) { return 1; }
    try { std::rethrow_exception(failure); }
    catch (int value) { return value == 7 ? 0 : 2; }
    catch (...) { return 3; }
}

```

这是单生产者、`join` 后观察的最小例子；整数异常只是测试标记，不是推荐的生产错误类型。线程创建和 `join` 自身的失败政策不在本例内。

`current_exception()` 不传播异常，但可能在内部尝试分配或复制异常；失败时可能改为保存 `bad_alloc`、复制失败的异常或 `bad_exception`，不能承诺它永不分配或始终保留原始错误。

N4950: propagation

工程上把可能分配的格式化、日志、上报推迟到协调者，并给这些动作定义后备路径：例如保留已有失败标记、设置预分配的简单计数 / 状态，或者按事先批准的策略终止；后备路径也不得再次依赖失败的报告设施。给未知报告函数加 `noexcept` 只会把逃逸异常变成终止，不会证明恢复成立。

T11 用人为编写的抛异常报告函数作受控终止反例，不是仓库生产实现的 bug 复现。

3. std::jthread

`std::jthread` 改善：

```

join
cooperative stop

```

`std::jthread` 不会把 `worker exception` 自动传播给创建线程。如果新线程中的 `invoke expression` 经异常退出，标准要求调用 `std::terminate()`；因此需要隔离 `worker failure` 时，仍必须在线程入口显式捕获并转换 / 保存失败。报告动作自身的失败也须按 §2 处理。N4950: `thread.jthread.cons/5`

4. std::promise / std::future

Future channel 保存值或异常；下面只说明通道使用方式，不是完整的线程入口隔离实现。

生产端片段：

```
try {
    promise.set_value(work());
} catch (...) {
    promise.set_exception(std::current_exception());
}
```

前提是 `promise` 有有效共享状态，且结果只被完成一次。`catch` 中的 `set_exception` 也可能因状态已就绪而抛出 `future_error`；它不是无条件可靠的报告后备。若工作在独立线程中执行，还须按 §2 把报告失败纳入边界。参见 N4950: `futures.promise`。

消费端：

```
auto value = future.get();
```

`get()` 观察共享状态并可能重新抛出已保存异常；其使用也要满足 `future` 自身的有效性前提。把异常存进通道不等于恢复了工作对象，更不免除共享状态和所有权的生命周期管理。

5. `std::async`

如果 `asynchronous operation` 通过 `exception` 失败，其 `associated future` 可以保存该异常，并在：

```
future.get()
```

时重新抛出。

因此: `execution boundary`

和: `observation boundary`

可能不在同一线程。

6. `std::exception_ptr`

通用模型：

```
catch exception
↓
current_exception()
↓
transport exception_ptr
↓
rethrow_exception()
```

适用于：

```
worker → coordinator
callback → event loop
background operation → observer
```

7. Coroutine Failure

协程函数体内逃出用户 handler 的异常，由协程变换中的对应处理路径调用：

```
promise_type::unhandled_exception()
```

task 的 promise 可选择保存异常、转换结果或终止，因此 task 类型本身必须声明传播与观察政策。

这不覆盖协程建立过程的一切失败：帧分配、参数副本或 promise 构造等不能统称为“都交给 unhandled_exception”。初始 await 的异常也有单独规则；是否采用 allocation-failure hook 要看 promise 定义。主张仅限定函数体路径，依据 N4950: `decl.fct.def.coroutine`。本章不增加 task runtime 实现，也不声称已验证这些路径的执行矩阵。

8. Coroutine 不是自动 Result Type

`co_await/co_return`:

不自动规定 failure semantics

failure 行为由：

```
awaiter
promise type
task abstraction
scheduler
```

共同决定。

因此不能说：coroutine 使用 exception

或者：coroutine 使用 expected

它取决于 coroutine abstraction。

二、取消、超时与关闭

9. Cancellation ≠ Failure

例如：

```
shutdown requested
user cancelled
deadline cancelled
task superseded
```

可能不是：component error

而是：control outcome

因此应尽量区分：

```
success
failure
cancelled
```

而不是：

```
success
error
```

两状态模型覆盖所有情况。

10. Timeout ≠ Cancellation ≠ Failure

三个概念描述不同观察，不是天然互斥的错误码：

概念	当前能知道什么	不能据此推断什么
Timeout	在约定截止期内没有观察到所需完成结果	操作没有执行、没有提交或之后不会完成
Cancellation request	请求停止 / 撤回后续工作	对方已停止、清理已完成或副作用已撤销
Failure	没有达到所要求的成功结果，或观察到具体失败	不变量必然已坏，或一定可以原样重试

取消确认、工作终态和外部提交应另外建模。一个远端操作可能先提交、再丢失响应，调用者超时后又请求取消；这些事实可以同时成立。恢复权限与完成不确定性继续见 §17–19。

11. Shutdown 不是 Error Storm

关闭过程中：

```
socket closed
queue rejected
operation cancelled
```

很多都可能是：expected shutdown outcomes

如果全部记录成 ERROR: observability signal distorted

成熟系统应区分：

```
failure during normal operation
vs
expected shutdown cancellation
```

三、集成和远端边界

12. Callback Boundary

框架调用：

```
callback(event);
```

如果 callback exception 逃入：

```
C library
GUI runtime
event loop
unknown framework
```

后果可能不可接受。

因此框架 boundary 应规定：

```
callback may throw?
callback must be noexcept?
framework catches?
```

不能留成隐式假设。

13. C ABI

对外：

```
extern "C" int process(...) {
    try {
        return run();
    } catch (...) {
        return kInternalError;
    }
}
```

核心规则：

不要让 C++ exception 穿越不受统一 C++ runtime/ABI 控制的 foreign boundary。

应转换为：

```
integer status
error struct
opaque error handle
```

14. Plugin ABI

插件环境还存在：

```
compiler mismatch
runtime mismatch
standard library mismatch
allocator mismatch
exception ABI mismatch
```

因此稳定 plugin ABI 通常应该拥有: explicit C-compatible boundary
或者严格控制整个 toolchain/runtime。

15. Process Boundary

Exception 无法跨 process 直接传播。

必须: serialize failure

因此错误协议需要:

```
stable error identity
versioning
context
retry semantics
```

这就是为什么: internal C++ type hierarchy
不能直接成为 RPC error protocol。

16. RPC Error Model

远程调用至少可能产生:

```
domain rejection
transport failure
timeout
cancellation
remote overload
remote internal failure
protocol incompatibility
```

不要压成: RPC failed

否则上层无法制定 recovery policy。

四、恢复协议与影响范围

17. Ambiguous Remote Completion

最重要的分布式错误之一:

```
request sent
remote side commits
response lost
caller times out
```

现在: caller sees failure

但: side effect may have happened

因此: retry

可能产生重复副作用。

18. Idempotency

自动重试要先证明“重复执行不会产生不允许的额外效果”，或有可靠证据证明前次未提交。否则需要协议层去重、状态查询或补偿，不能只凭 timeout 再发一次。

Idempotency key、request ID 和 transaction token 只是协议材料：接收方如何绑定请求与副作用、保留记录多久、如何处理并发重复及确认丢失，才决定它们能保证什么。即使操作幂等，仍须检查 §19 的截止期、预算和关闭状态。

19. Retry Budget

Retry 不能无限：

```
retry
→ overload
→ more timeout
→ more retry
```

形成 retry storm。

成熟系统通常定义：

```
deadline
max attempts
backoff
jitter
retryable categories
shutdown awareness
```

这些都属于: recovery policy

而不是底层 transport 自动行为。

20. Supervisor

并发系统中: worker

应尽量只: detect/report local failure

更高层 supervisor:

```
restart?
drop task?
stop subsystem?
stop process?
```

这样 recovery authority 清晰。

21. Failure Domain Hierarchy

一个服务可以定义：

```
operation
↓
task
↓
worker
↓
subsystem
↓
process
↓
service
```

每种 error 必须知道：maximum blast radius

例如：

```
malformed packet
  → packet

worker unexpected exception
  → worker/task

central invariant corruption
  → process
```

22. Logging Boundary

一个 failure 如果沿五层传播：不要每层都 ERROR log

否则：

```
one failure → five error records
```

建议：

```
lower layers:
  preserve/enrich context

decision boundary:
  emit final log/metric
```

五、审查与回查

23. FM-8 Review Checklist

先用公共 C1-C8 合同检查完整操作，再用以下问题回查本章机制。

- [] exception 是否可能逃出 thread entry?
- [] async failure 是否有明确 observation point?
- [] exception_ptr ownership/lifetime 是否明确?
- [] coroutine promise 的 unhandled_exception policy 是否明确?
- [] cancellation 是否与 failure 分离?
- [] timeout 是否可能存在 ambiguous completion?
- [] shutdown cancellation 是否错误计为系统 failure?
- [] callbacks 是否有 exception contract?
- [] C/plugin ABI 是否 containment exception?
- [] RPC error taxonomy 是否稳定?
- [] retry 是否验证 idempotency?
- [] retry 是否有 deadline/budget/backoff?
- [] failure domain 是否限定?
- [] supervisor 是否拥有正确 recovery authority?
- [] log 是否集中在 decision boundary?

24. FM-8 核心不变量

Thread、ABI、process、RPC 都是 failure boundaries。

Cancellation、timeout 和 failure 应分别建模。

Retry 属于 policy，必须建立在 idempotency 和 deadline 之上。

Remote timeout 不能证明 remote operation 没有执行。

并发系统应明确 supervisor 和 failure domain。

FM-9 · 项目失败契约与验证

SOURCE SNAPSHOT CONTENT · 45b305e

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

C++23 失败语义工程手册

[返回 FM 导航](#) · [上一章：FM-8 · 术语与审查约定](#)

阅读入口

把 FM-0 ~ FM-8 的机制压缩为项目可以选择、填写和验证的合同。这里的 **must/default** 是待采纳政策，不是 C++ 的统一语言要求。

主阅读线。 §1-3 → §9-17 → §21-30；需要时回查各机制政策及最后总图。

失败契约。项目逐项采纳错误通道、状态、所有权和恢复权限；模板要包含提交前后及完成未知，并给每个重要主张配对对应判据和未测项。

证据边界。故障注入清单是待项目执行的验证设计；FM 样例通过不等于某项目已验证，也不表示已部署 CI 或获得正式基线批准。无 **fm-test** 标记的片段按上下文阅读，不自动视为完整实验。

本章目录

CHAPTER READING PATH · 原文章内导航

- 一、项目选择与责任
- 二、通道与错误域政策
- 三、状态、所有权与边界政策
- 四、恢复与可观测性政策
- 五、验证工具与证据
- 六、可裁剪模板与回查
- 最终总图

原 § 编号用于稳定回查；组标题只组织阅读，不新增机制范围。

一、项目选择与责任

0. 文档定位

FM-0 ~ FM-8 描述语言与系统机制。

本章是待采纳的工程 **profile 模板**，不是已批准项目基线。下文“必须 / 不得 / 默认”等政策措辞仅约束明确采纳该 **profile** 的项目；语言与库规则另有标准依据。解析拒绝倾向 **expected**、析构不传播异常等默认策略，不能冒充所有 C++ 程序统一适用的规范事实。

FM-9 的目标是：

将这些规则压缩成一个项目真正可以执行的 **Failure Profile**。

如果没有统一 **profile**，工程最终容易变成：

```
模块 A 用 exception
模块 B 用 bool
模块 C 用 expected<string>
模块 D 用 errno
模块 E catch(...)
模块 F abort()
```

每种单独看可能合理，

组合后却没有一致系统语义。

1. Failure Profile 应解决什么

项目必须统一回答：

```
什么是 domain outcome?
什么是 recoverable failure?
什么是 programmer error?
哪些 API 可以 throw?
哪些 API 使用 expected?
哪些 boundary 禁止 exception?
哪些 invariant release 也必须检查?
哪些操作提供 strong/basic guarantee?
哪些 failure 可以 retry?
何时 terminate?
```

2. 推荐项目分类

以下是项目的处置标签，不是互斥且完整的原因分类。**P** 描述责任，**F** 描述严重性 / 终局处置；同一事件可能同时是 **P** 与 **F**。选定恢复 / 终止策略时应保留来源、频率和状态等独立信息。

D — Domain Outcome
R — Recoverable Operational Failure
P — Programmer / Contract Failure
F — Fatal System Failure

D — Domain Outcome

例如：

not found
queue empty
optional field missing
EOF

默认：

optional
bool
enum

R — Recoverable Failure

例如：

invalid external input
I/O failure
network timeout
configuration rejection
resource unavailable

默认：

expected<T, E>
error_code
exception where project policy allows

P — Programmer Error

例如：

precondition violation
invalid internal index
broken state-machine invariant
double ownership

默认：

assert
production fatal check where necessary

不应默认：expected<..., Bug>

F — Fatal Failure

例如：

```
central invariant corruption
unrecoverable process state
exception escaping noexcept
critical runtime corruption

默认:

terminate
abort
process-level fail-fast policy
```

3. 推荐 Layer Policy

可以定义：

Layer	默认 Failure Style
Parser / Validation	std::expected<T, E>
Domain lookup	std::optional<T> / expected
Core trusted algorithms	Preconditions + assertions
Resource wrappers	RAII + constructor/factory
OS abstraction	error_code / expected
Application orchestration	expected 或 exception，根据项目统一政策
Thread entry	Catch/contain unexpected exception
C/plugin ABI	Explicit status/error object
RPC boundary	Serialized structured errors
Fatal invariant boundary	Fail-fast

二、通道与错误域政策

4. Exception Policy

项目必须明确选择。

例如：

```
Exceptions:
    enabled

Allowed:
    construction failure
    rare infrastructure failure
    orchestration propagation
```

FM-9 · 4. Exception Policy · continued

Not allowed:

- parser rejection
- normal lookup miss
- hot-path control flow

Must not cross:

- thread entry
- C ABI
- plugin ABI
- RPC

或者:

Application code:

- exception-free by policy

Third-party exceptions:

- caught and translated at adapter boundary

关键不是哪一种绝对最好,

而是:

policy 必须一致。

5. expected Policy

推荐规定:

Use expected when:

- failure is expected at runtime
- caller can meaningfully branch/recover
- failure belongs to API result domain

避免: every function returns expected

特别是: private helper whose preconditions are already established

不需要重新包装 internal impossible state。

6. Error Type Policy

推荐:

```

error identity:
    enum / structured code

diagnostic context:
    structured fields

human text:
    produced at diagnostic boundary

```

避免默认：

```
std::expected<T, std::string>
```

成为跨大型系统统一协议。

7. Error Namespace

可以：

```

namespace parser {

enum class ErrorCode {
    truncated,
    invalid_header,
};

struct Error {
    ErrorCode code;
    std::size_t offset;
};

}

```

不同 subsystem：

```

parser::Error
storage::Error
network::Error

```

不要创建一个无限膨胀：

```

enum class GlobalError {
    ...
};

```

8. Error Translation Policy

规定：

只有跨越 abstraction boundary 时翻译。

例如：

```
socket error
↓
HTTP client error
↓
repository unavailable
```

不要：

```
每一个函数层次
↓
创建新 Error wrapper
```

三、状态、所有权与边界政策

9. State Guarantee Policy

本节是待项目采纳的 profile 建议，不是所有 C++ API 的语言要求。建议每个重要的 mutating API 分别声明：

维度	合同内容
失败后状态	strong、basic、明确部分进度或其他具体语义
异常传播	是否允许异常越界；是否具有 noexcept 规格
资源与所有权	失败后资源属于谁，哪些清理动作有保证
终局策略	无法继续时升级、重启或终止的范围

例如 update(Config) 返回 expected<void, UpdateError> 时，仍须单独说明“失败时旧配置不变”，或者“保留哪些部分进度且哪些不变量继续成立”。不从返回类型推出 strong，不从 noexcept 推出 no-fail，也不把 termination 排为状态保证的一级。

精确定义及 commit 条件见 FM-4 §4 ~ §5。

10. Ownership Failure Policy

任何 transferring API 都必须说明：

```
ownership before call
ownership after success
ownership after failure
```

例如：enqueue(Job job)

和：try_enqueue(Job& job)

具有完全不同的 failure ownership semantics。

11. noexcept Policy

推荐:

```

Destructors:
    must not propagate exceptions

Move operations:
    noexcept when semantically true

Swap:
    non-throwing and truly non-failing under commit preconditions

Cleanup:
    noexcept

Leaf arithmetic / trivial accessors:
    noexcept where contract truly permits

Do not:
    add noexcept merely for performance

```

12. Assertion Policy

项目可以定义两种检查；这里的 `development/production` 是部署政策，不是语言按构建名称自动切换的行为。标准 `assert` 是否启用取决于包含头文件时的 `NDEBUG`，见 [FM-1 §13](#)。

Development Assertion

```

enabled in the project's chosen development configuration
internal reasoning
cheap diagnostics

```

例如:

```
assert(index < size);
```

Production Fatal Check

用于:

```

release must enforce
continuing could corrupt persistent/external state
security-sensitive invariant
central internal invariant

```

应由项目统一宏/函数提供:

```

CHECK
ENSURE
FATAL_IF

```

具体名字不是重点。

13. UB Policy

项目原则: UB is never a failure-handling strategy.

应该:

```
validate untrusted input
assert trusted preconditions
use sanitizers
use static analysis
use ownership/lifetime discipline
```

目标是: prevent UB

而不是: recover from UB

14. Boundary Policy

必须列出所有重要边界:

```
thread
coroutine task
event loop
C ABI
plugin ABI
IPC
RPC
process main
```

每个边界写明:

```
incoming failure forms
outgoing failure forms
exception policy
logging responsibility
failure domain
```

四、恢复与可观测性政策

15. Retry Policy

项目统一规定: retry only when category is retryable

同时检查:

```
idempotency
deadline
attempt count
backoff
jitter
shutdown
system load
```

禁止基础库：`while (!success) retry();`

16. Timeout Policy

Timeout 应明确：`deadline expired`

但不能自动断言：`remote operation failed before commit`

对于有外部副作用的操作：`timeout semantics`

必须说明：

```
definitely not committed
possibly committed
definitely committed
```

如果无法知道：`ambiguous`

应直接建模。

17. Cancellation Policy

推荐作为独立 outcome：

```
success
failure
cancelled
```

尤其：

```
worker shutdown
async operation
coroutine
RPC
```

避免把正常 shutdown 变成错误风暴。

18. Logging Policy

基本原则：

```
errors are propagated many times
but usually logged once
```

通常：

```
detection layer:
    produce structured error

intermediate layer:
    enrich context

recovery/terminal boundary:
    log
```

19. Metrics Policy

Metrics 应按：

```
error category
operation
component
recovery action
```

聚合。

避免使用：raw error message

作为 metric label,

否则容易产生：unbounded cardinality

20. Diagnostics Policy

最终 diagnostics 可以组合：

```
error code
cause chain
operation
resource identity
request/job ID
location
context
```

但要明确：

```
secret redaction
PII policy
size limits
```

五、验证工具与证据

21. Testing Failure Paths

Failure path 必须是: first-class test target

而不是只测试 success path。

至少包括:

```
invalid input
allocation/resource failure where injectable
I/O error
timeout
cancellation
partial progress
throwing T in generic code
thread exception
shutdown race
```

22. Fault Injection

高质量组件应允许测试:

```
fail allocation N
fail write N
timeout operation N
disconnect after commit
throw during element move
```

这样才能验证:

```
strong/basic guarantee
cleanup
retry
failure domain
```

23. Sanitizers

Failure Model 无法替代:

```
ASan
UBSan
TSan
```

这些主要帮助发现:

```
memory safety
UB
data race
```

它们属于: defect detection infrastructure
而不是 runtime recovery system。

24. Static Analysis

推荐结合:

```
clang-tidy
compiler warnings
lifetime-oriented checks
nodiscard
concepts
type system
```

尽可能将:

```
failure handling omission
invalid state
ownership bug
```

提前到开发阶段。

25. `[[nodiscard]]`

对必须观察的结果可使用 `[[nodiscard]]`:

```
[[nodiscard]]
std::expected<void, Error> save();
```

若写成:

```
save();
```

C++23 的规定是建议实现对这类被丢弃的结果发出警告，不是必须拒绝编译，也不是结果一定会被处理的静态证明。显式转换为 `void` 又属于不同情况。项目可通过告警选项和评审加强执行，但必须另外说明，不能归到语言强制保证。N4950: `dcl.attrnodiscard`

六、可裁剪模板与回查

26. Public API Failure Documentation Template

本模板是公共 C1-C8 的唯一可填写版本；按 API 风险裁剪，不机械要求所有局部函数都有 RPC 或重试字段。

```
### Failure Contract

Operation / observation scope:
- API、版本、受保护状态及外部副作用范围。
```

FM-9 · 26. Public API Failure Documentation Template · continued

C1 — Inputs and success:

- 前置条件及建立者；合法领域结果；成功后置条件。

C2 — Failure set and channels:

- 每个失败来源、检测点、返回错误 / 异常 / 终止。
- 错误转换及诊断路径自身能否失败。

C3 — State transitions and failure postconditions:

- 准备阶段、提交点、返回 / 清理阶段。
- 每个失败分支的状态、不变量、部分进度。
- 明确未提交 / 已提交 / 完成未知；调用者如何得知。

C4 — Ownership and cleanup:

- 调用前、成功、失败、取消之后资源各属谁。
- 析构后备与需要显式观察的 `finish/close/commit`。

C5 — Composition:

- T/E、`allocator`、参数、回调、临时对象、返回与清理的前提。
- 异常传播保证与 `no-fail`/状态保证分别声明。

C6 — Recovery and containment:

- 恢复 / 补偿 / 升级 / 终止权限、`failure domain`。
- 线程安全、同步与错误观察点、报告失败后备。

C7 — Retry and control outcomes:

- 幂等 / 去重 / 确认依据、截止期、预算、关闭状态。
- 取消请求与确认、超时、完成未知的处置。

C8 — Verification:

- 命题、判据、注入点、错误变体、预期与实际结果。
- 工具链、平台、SKIP、不适用项、未覆盖边界。

27. Subsystem Failure Profile Template

Failure Profile**### Error Types**

- ...

Exception Policy

- ...

Validation Boundary

FM-9 · 27. Subsystem Failure Profile Template · continued

```
- ...

### Recovery Boundary

- ...

### Failure Domain

- ...

### State Guarantees

- ...

### Retry / Timeout

- ...

### Logging / Metrics

- ...

### Fatal Invariants

- ...
```

28. Project-Wide Baseline

推荐默认：

```
External input
  → validate

Normal absence
  → optional

Expected recoverable failure
  → expected

OS/native failure
  → error_code at low layer
  → translate when abstraction changes

Rare deep failure
  → exception only if project profile permits

Programmer error
  → assert / fatal check
```

FM-9 · 28. Project-Wide Baseline · continued

Unrecoverable invariant loss

→ fail-fast

Owned resources (release responsibility)

→ RAII

Mutating operations

→ document strong/basic guarantee

Move/swap/cleanup

→ noexcept where semantically true

Threads/ABI/RPC

→ explicit failure boundaries

29. Project Anti-Patterns

禁止形成: exceptions everywhere

或者: expected everywhere

或者: no exceptions at all costs

这些都是机制驱动设计。

正确顺序:

failure semantics

↓

responsibility

↓

state guarantee

↓

boundary

↓

frequency

↓

transport mechanism

30. Project Review Checklist

先用公共 C1-C8 合同检查完整操作，再用以下问题回查本章机制。

- ☐ 项目是否有统一 failure taxonomy?
- ☐ domain outcome / operational failure / programmer error 是否分离?
- ☐ exception policy 是否明确?
- ☐ expected 使用边界是否明确?
- ☐ errors 是否结构化?
- ☐ error translation 是否按 abstraction boundary?
- ☐ public mutating APIs 是否定义 state guarantee?

FM-9 · 30. Project Review Checklist · continued

- [] ownership failure semantics 是否明确？
- [] noexcept policy 是否明确，且未与 no-fail 或 strong/basic 混为一谈？
- [] assert 与 production fatal check 是否区分？
- [] UB 是否被明确视为 defect 而非 failure channel？
- [] thread / ABI / RPC boundary 是否明确，并覆盖错误报告自身的失败？
- [] cancellation / timeout / shutdown 是否单独建模？
- [] retry 是否有 idempotency + deadline + budget？
- [] logging 是否集中在 recovery/terminal boundary？
- [] error metrics 是否避免高 cardinality？
- [] failure paths 是否有测试？
- [] 是否支持关键 fault injection？
- [] sanitizer/static analysis 是否进入 CI？

31. FM-9 核心不变量

项目采纳 profile 时，应固定错误通道、状态保证、所有权、恢复权限和验证责任。选择机制在理解失败语义之后，不能从 expected 或 noexcept 倒推保证。

责任、可恢复性和终局处置分别记录：程序缺陷可能同时需要致命处置，运行时失败也未必可恢复。它们不是互斥的三个错误类别。线程、ABI、RPC 的转换以及 retry、timeout、cancellation、shutdown 均要按系统合同分析，而不是留给底层自行猜测。

Modern C++ Failure Model — 最终总图

把一次操作沿同一条链审查；状态、传播和处置分轴记录，不把不可恢复放进 strong/basic 的等级中：

```

Operation + Preconditions + Success Postcondition
      ↓
Possible outcomes / detection
      ↓
Representation → translation → propagation
      ↓
State transition audit
  ├── protected state: unchanged / valid-but-changed / partial / unspecified
  ├── resources: ownership / cleanup / outstanding obligation
  ├── commit: before / after / observer cannot determine
  └── exception channel: allowed / non-propagating boundary
      ↓
Recovery authority + failure domain
  ├── reject / recover / compensate / retry under protocol
  └── escalate / terminate if continued execution is not trustworthy
      ↓
Observable evidence + limits

```

责任、发生频率、严重性和恢复权限分别作为决策输入。错误传播只是把事实带到决策点；资源清理不代替回滚，捕获不代替恢复，终止不代替 no-fail 提交。

全系列最终回查顺序是：FM-0 定义模型，FM-1 建立输入前提，FM-2/FM-3 选择值或异常通道，FM-4 分析状态，FM-5/FM-6 核查泛型与生命周期，FM-7 转换错误域，FM-8 限定传播与恢复，最后在本章记录项目选择和证据。具体问题入口见 **FM** 总导航，共同词义见系列阅读约定。

这是待项目选择和验证的工程框架，不是对全部代码的技术认证，也不因本轮系列整理而成为已批准项目规范。